Towlion Platform

Documentation

None

None

Table of contents

1. Getting Started 3
   - 1.1 Towlion Platform 3
   - 1.2 Scope and Design Boundaries 5
   - 1.3 Deployment Tutorial 7
2. Architecture 13
   - 2.1 Architecture 13
   - 2.2 Application Specification 18
   - 2.3 Server Contract 22
3. Deployment 28
   - 3.1 Deployment 28
   - 3.2 Self-Hosting 31
   - 3.3 Preview Environments 35
   - 3.4 Deploy Prompt Template 37
   - 3.5 Self-Hosted GitHub Actions Runners 38
4. Operations 41
   - 4.1 Towlion CLI 41
   - 4.2 Secrets Reference 43
   - 4.3 Observability 46
   - 4.4 Troubleshooting 49
   - 4.5 Runbooks 52
5. Project 61
   - 5.1 Ecosystem 61
   - 5.2 Governance 64
   - 5.3 Onboarding 67
   - 5.4 Decisions 69

1. Getting Started

1.1 Towlion Platform

Self-hosted GitHub-native micro-PaaS for deploying small web applications.

Deploy full web applications to your own server directly from GitHub. No custom dashboards, no complex control planes — just GitHub repositories, Actions, and a single server.

1.1.1 What is Towlion?

Most platforms require a central control plane:

```
CLI → platform API → Kubernetes → application containers
```

Towlion takes a different approach. **GitHub becomes the control plane.**

```
GitHub repository → GitHub Actions → SSH deploy → server runtime
```

Each repository contains everything required to deploy itself. Push code, and your application is live.

See Scope and Design Boundaries for what the platform does and does not do.

1.1.2 Key Features

- **GitHub-native deployment** — no custom CLI or dashboard needed
- **Self-hosted infrastructure** — runs on your own Debian server
- **Multiple applications on one server** — shared PostgreSQL, Redis, MinIO
- **Automatic TLS certificates** — via Caddy and Let's Encrypt
- **Background workers** — Celery + Redis for async tasks
- **Object storage** — S3-compatible via MinIO
- **Preview environments** — temporary deployments for pull requests
- **Fork-based self-hosting** — anyone can fork and deploy

1.1.3 Platform Stack

Layer	Technology	Purpose
OS	Debian	Host operating system
Containers	Docker + Compose	Runtime and orchestration
Reverse proxy	Caddy	TLS + routing
Backend	FastAPI (Python)	Application API
Frontend	Next.js (React, TypeScript)	Web interface
Database	PostgreSQL	Persistent data
Cache / Queue	Redis	Caching + job queues
Object storage	MinIO	S3-compatible storage
Background jobs	Celery	Async processing
CI/CD	GitHub Actions	Automated deployments

1.1.4 Why Towlion?

Most cloud platforms are designed for large-scale applications. Towlion focuses on a different use case: **small, deployable applications**.

The goal is to make it easy to:

- Build small web tools
- Deploy them quickly
- Self-host them on your own server
- Share them with others through GitHub

1.1.5 License

MIT License

1.2 Scope and Design Boundaries

1.2.1 What Towlion Is

- A **single-server micro-PaaS** for deploying small web applications
- **GitHub as the control plane** — no custom dashboard, CLI, or API
- **Docker Compose** for container orchestration, **Caddy** for reverse proxy and automatic TLS
- **Fork-to-deploy** model — fork a template repo, configure secrets, push to deploy
- **Opinionated stack** — FastAPI, PostgreSQL, Redis, MinIO, Celery

1.2.2 Design Boundaries

These are intentional constraints, not missing features. They keep the platform simple and predictable.

Infrastructure

Boundary	Detail
Single server only	No clustering, no multi-server, no load balancing. Your server is a single point of failure by design.
Debian Linux only	The bootstrap script targets Debian 12. Other distributions are untested.
No automated server provisioning	You create and manage your own server. The bootstrap script configures it, but you are responsible for procurement and access.
Fixed resource limits	Static CPU and memory limits per container. No auto-scaling.

Deployment

Boundary	Detail
Brief downtime during deploys	`docker compose up -d --build` rebuilds and restarts containers. Caddy returns 502 during the gap. This is typically a few seconds.
No built-in test pipeline	The deploy workflow builds and deploys. It does not run your test suite — add that step yourself if needed.
Push-to-main deploys	No staging environment, no approval gates, no canary or blue-green deploys. Merging to `main` deploys to production.

Operations

Boundary	Detail
No high availability	Single PostgreSQL, Redis, and MinIO instances. No replication or automatic failover.
5-minute alert granularity	Health checks run via cron every 5 minutes. This is not real-time monitoring.
7-day backup retention	Daily `pg_dump` with 7-day retention. Recovery is manual. There is no disaster recovery plan beyond these backups.
No multi-tenant management	Each fork is independent. There is no shared admin panel or centralized management across apps.

Technology

Boundary	Detail
Not Kubernetes	Docker Compose only. If you need pods, services, ingress controllers, or declarative infrastructure, use Kubernetes.
Opinionated stack	FastAPI + PostgreSQL + Redis + MinIO + Caddy. Other frameworks and databases are outside the supported path.
No container registry	Images are built on the server from source code. There is no registry push/pull step.

1.2.3 When to Use Something Else

If you need...	Consider instead
Multi-server / high availability	Kubernetes, Docker Swarm, Nomad
Auto-scaling	AWS ECS, Cloud Run, Fly.io
Managed databases	AWS RDS, DigitalOcean Managed DB
Zero-downtime deploys	Kubernetes rolling updates, blue-green deploy tooling
Large-scale traffic	Cloud-native platforms with load balancers
Non-Python backends	Dokku, Coolify, CapRover

1.2.4 Sweet Spot

Towlion is built for indie developers, small SaaS products, personal tools, hobby projects, and AI-generated applications. If your app serves a handful of users, runs comfortably on a single server, and you want to own your infrastructure without managing Kubernetes, Towlion is a good fit.

1.3 Deployment Tutorial

This tutorial walks you through deploying a Towlion application from fork to running service. By the end, you will have a live application on your own server with automatic TLS, a database, and continuous deployment from GitHub.

> **Before you start**
>
> This is a hands-on guide with concrete commands. For background on *why* the platform works this way, see Self-Hosting for the fork model and Deployment for pipeline internals.

1.3.1 Prerequisites

You will need:

- A **GitHub account**
- A **Debian server** (VPS from any provider — Hetzner, DigitalOcean, Linode, etc.)
- A **domain name** you control (for DNS configuration)
- A local machine with **Git** and **SSH** installed

Resource	Minimum
CPU	2 cores
RAM	4 GB
Disk	50 GB

1.3.2 Step 1: Fork the app repository

Go to the application repository on GitHub (for example, towlion/app-template) and click **Fork**.

Then clone your fork locally:

```
git clone git@github.com:YOUR_USERNAME/app-template.git
cd app-template
```

> **Tip**
>
> If you are creating a new app rather than deploying an existing one, use the **Use this template** button on towlion/app-template instead of forking. This gives you a clean commit history.

1.3.3 Step 2: Provision a server

Create a Debian server from your preferred provider. Make sure:

- Ports **22**, **80**, and **443** are open in the firewall
- You can SSH in as root (or the user your provider creates)

Verify access:

```
ssh root@YOUR_SERVER_IP
```

You should see a shell prompt. If this works, you are ready to bootstrap.

Automated provisioning

Instead of creating a server manually, you can use towlion/infra to provision a Debian server with a single command:

```
./towlion-infra init --provider digitalocean
./towlion-infra apply --domain example.com
```

This creates a server with the correct firewall rules, a data volume, SSH key, and optional DNS — ready for bootstrap. See the infra README for setup and usage.

1.3.4 Step 3: Bootstrap the server

SSH into your server as root. Download and run the bootstrap script — no `git` required:

```
ssh root@YOUR_SERVER_IP
mkdir -p /tmp/platform && curl -fsSL https://github.com/towlion/platform/archive/refs/heads/main.tar.gz \
  | tar xz --strip-components=1 -C /tmp/platform
sudo ACME_EMAIL=you@example.com bash /tmp/platform/infrastructure/bootstrap-server.sh
```

Tip

Set `OPS_DOMAIN=ops.example.com` to also enable the Grafana monitoring dashboard:

```
sudo ACME_EMAIL=you@example.com OPS_DOMAIN=ops.example.com bash /tmp/platform/infrastructure/bootstrap-server.sh
```

Verify the bootstrap was successful:

```
/opt/platform/infrastructure/verify-server.sh
```

All checks should pass. The script creates:

```
/data/              # Persistent data (postgres, redis, minio, caddy, loki, grafana)
/opt/platform/      # Platform services and config
/opt/apps/          # Application deployments
```

1.3.5 Step 3.5: Clone your app on the server

The deploy workflow runs `git pull` (not `git clone`), so the app must be pre-cloned on the server. SSH in as the `deploy` user:

```
ssh deploy@YOUR_SERVER_IP
cd /opt/apps
git clone git@github.com:YOUR_USERNAME/app-template.git my-app
cd my-app
```

Provision per-app database and storage credentials:

```
sudo /opt/platform/infrastructure/create-app-credentials.sh my-app
```

Create the deploy environment file from the template:

```
cp deploy/env.template deploy/.env
```

The deploy workflow will auto-update `deploy/.env` with the correct credentials on the next push.

CLI shortcut

The `towlion create` command handles repo creation, credential provisioning, cloning, and env setup in one step:

```
towlion create my-app
```

See the CLI Reference for installation and configuration.

Warning

The `deploy` user needs SSH access to your GitHub repo to `git pull`. Add the deploy user's public key (`/home/deploy/.ssh/id_ed25519.pub`) as a deploy key on your GitHub repository, or use HTTPS cloning with a personal access token.

1.3.6 Step 4: Configure DNS

Go to your domain registrar or DNS provider and add an **A record** pointing to your server:

```
Type: A
Name: app          (or your chosen subdomain)
Value: YOUR_SERVER_IP
TTL: 300
```

For example, if your domain is `example.com` and your server IP is `203.0.113.42`:

```
A record: app.example.com -> 203.0.113.42
```

Verify DNS propagation:

```
dig +short app.example.com
```

Expected output:

```
203.0.113.42
```

Tip

DNS propagation can take a few minutes to a few hours. Wait until `dig` returns your server IP before proceeding.

1.3.7 Step 5: Configure GitHub secrets

In your forked repository on GitHub, go to **Settings > Secrets and variables > Actions** and add the following repository secrets:

Secret	Example value	Description
`SERVER_HOST`	`203.0.113.42`	Your server's IP address
`SERVER_USER`	`deploy`	SSH username on the server
`SERVER_SSH_KEY`	*(private key contents)*	SSH private key for deployment
`APP_DOMAIN`	`app.example.com`	Domain pointing to your server

Note: Database and storage credentials are auto-generated by the bootstrap script on the server. You do not need to create them as GitHub secrets.

Optionally, add `PREVIEW_DOMAIN` (e.g., `example.com`) to enable preview environments for pull requests.

CLI shortcut

The `towlion secrets` command sets all required GitHub secrets using your local config and SSH key:

```
towlion secrets my-app app.example.com
towlion secrets my-app app.example.com --preview-domain example.com
```

Generate a deploy SSH key

Create a dedicated key pair for deployment:

```
ssh-keygen -t ed25519 -f ~/.ssh/deploy_key -N ""
```

Add the **public** key to your server:

```
ssh-copy-id -i ~/.ssh/deploy_key.pub deploy@YOUR_SERVER_IP
```

Copy the **private** key contents into the `SERVER_SSH_KEY` secret:

```
cat ~/.ssh/deploy_key
```

Paste the full output (including the `-----BEGIN` and `-----END` lines) into the secret value field on GitHub.

1.3.8 Step 6: Deploy

Push a commit to the `main` branch to trigger deployment:

```
git push origin main
```

GitHub Actions picks this up automatically. Go to the **Actions** tab in your repository to watch the workflow run.

```
Push to main
     |
     v
GitHub Actions
     |
     +-- Run tests
     +-- SSH into server
     +-- Pull latest code
     +-- Build containers
     +-- Start services
     +-- Run database migrations (inside container)
     +-- Health check
```

The workflow typically completes in 2-5 minutes.

CLI shortcut

You can also trigger deploys from the command line:

```
towlion deploy my-app
```

Tip

If the workflow does not appear, check that the `.github/workflows/deploy.yml` file exists in your repository. Repositories created from the app template include this file by default.

1.3.9 Step 7: Verify

Once the workflow succeeds, check your application is running.

Test the health endpoint:

```
curl https://app.example.com/health
```

Expected response:

```
{"status": "ok"}
```

CLI shortcut

```
towlion health my-app
```

Open `https://app.example.com` in your browser. You should see your application with a valid TLS certificate (Caddy provisions this automatically via Let's Encrypt).

Your application is now live.

1.3.10 Updating your app

To deploy changes, commit and push to `main`:

```
git add .
git commit -m "feat: add new feature"
git push origin main
```

GitHub Actions runs the deployment pipeline automatically. Your application will be briefly unavailable during the rebuild (typically a few seconds).

To pull upstream changes from the original repository:

```
git remote add upstream https://github.com/towlion/app-template.git
git fetch upstream
git merge upstream/main
git push origin main
```

1.3.11 Troubleshooting

DNS not resolving

Symptom: `dig +short app.example.com` returns nothing.

Fix: Wait for DNS propagation (up to 48 hours in rare cases). Verify the A record is set correctly in your DNS provider's dashboard. Try flushing your local DNS cache:

```
# macOS
sudo dscacheutil -flushcache

# Linux
sudo systemd-resolve --flush-caches
```

SSH key rejected

Symptom: GitHub Actions workflow fails with `Permission denied (publickey)`.

Fix: Verify the `SERVER_SSH_KEY` secret contains the full private key including header and footer lines. Ensure the corresponding public key is in `~/.ssh/authorized_keys` on the server. Check that the key format is correct:

```
# The secret should start with:
-----BEGIN OPENSSH PRIVATE KEY-----

# And end with:
-----END OPENSSH PRIVATE KEY-----
```

Health check fails

Symptom: Deployment completes but `curl https://app.example.com/health` returns an error.

Fix: SSH into the server and check container status:

```
ssh deploy@YOUR_SERVER_IP
docker compose ps
```

All services should show `Up` status. Check application logs:

```
docker compose logs app --tail 50
```

CLI shortcut

```
towlion status        # container states at a glance
towlion logs my-app   # tail app logs
```

Common causes:

- Database migration failed — run `docker compose exec app alembic -c app/alembic.ini upgrade head` to retry migrations, and check `docker compose logs app` for errors
- Missing environment variable — verify all secrets are set in GitHub
- Port conflict — ensure no other service is using ports 80 or 443

Containers not starting

Symptom: `docker compose ps` shows containers in `Restarting` or `Exit` state.

Fix: Check the logs for the failing container:

```
docker compose logs postgres --tail 50
docker compose logs app --tail 50
```

If PostgreSQL fails to start, verify the `/data/postgres` directory exists and has correct permissions:

```
ls -la /data/postgres
```

TLS certificate not provisioning

Symptom: Browser shows a certificate warning when visiting your domain.

Fix: Caddy provisions TLS certificates automatically, but requires:

1. DNS is correctly pointing to your server
2. Ports 80 and 443 are open and reachable from the internet
3. The domain is set correctly in your app configuration

Check Caddy logs:

```
docker compose logs caddy --tail 50
```

For more details on the deployment pipeline, see Deployment. For the full list of application requirements, see the App Specification.

2. Architecture

2.1 Architecture

2.1.1 Overview

The Towlion platform runs on a single Debian server. Applications run as Docker containers and share a set of core infrastructure services.

```
graph TB
    User[User Browser] -->|HTTPS| Caddy

    subgraph Server["Debian 12 Server"]
        subgraph Docker["Docker / towlion network"]
            Caddy["Caddy :80/:443"]

            Caddy --> App1["App 1 :8000"]
            Caddy --> App2["App 2 :8000"]
            Caddy --> App3["App 3 :8000"]
            Caddy --> Grafana["Grafana :3000"]

            App1 --> Postgres[("PostgreSQL :5432")]
            App2 --> Postgres
            App3 --> Postgres
            App1 --> Redis[("Redis :6379")]
            App1 --> MinIO["MinIO :9000"]

            Promtail["Promtail"] --> Loki["Loki :3100"]
            Loki --> Grafana

            Prometheus["Prometheus :9090"] -.->|optional| Grafana
            cAdvisor["cAdvisor :8080"] -.->|optional| Prometheus
            NodeExp["Node Exporter :9100"] -.->|optional| Prometheus
        end
    end
```

Dashed lines indicate optional metrics services (enabled via `COMPOSE_PROFILES=metrics`).

2.1.2 Technology Stack

Infrastructure

- **Debian** — host operating system
- **Docker** — container runtime
- **Docker Compose** — service orchestration

Networking

- **Caddy** — reverse proxy with automatic TLS via Let's Encrypt

Application

- **FastAPI** — Python backend framework
- **SQLAlchemy** — ORM
- **Pydantic** — data validation
- **Alembic** — database migrations

Frontend

- **Next.js** — React framework
- **TypeScript** — type-safe frontend code

Data

- **PostgreSQL** — primary database
- **Redis** — caching and job queues
- **MinIO** — S3-compatible object storage

Background Jobs

- **Celery** — async task processing (backed by Redis)

CI/CD

- **GitHub Actions** — automated builds and deployments

Optional Observability

- **Grafana** — dashboards
- **Loki** — log aggregation

2.1.3 GitHub as the Control Plane

Traditional PaaS platforms use a dedicated control plane:

```
CLI → Control plane → Kubernetes → containers
```

Towlion replaces this with GitHub:

```
GitHub repo → GitHub Actions → SSH deployment → Docker runtime
```

GitHub provides:

- CI/CD (Actions)
- Configuration storage (Secrets)
- Access control (repository permissions)
- Workflow orchestration (Actions workflows)

This eliminates the need for custom deployment dashboards or orchestration systems.

2.1.4 Multi-Application Hosting

A single server hosts multiple applications. Each application runs in its own container, sharing the core infrastructure services.

```
server
 ├── caddy
 ├── postgres
 ├── redis
 ├── minio
 ├── uku-app
 ├── timer-app
 └── lyrics-app
```

2.1.5 Domain Routing

Each application gets its own subdomain. Caddy routes traffic to the correct container.

```
uku.towlion.com   → uku-app container
timer.towlion.com → timer-app container
lyrics.towlion.com → lyrics-app container
```

Example Caddy configuration:

```
uku.towlion.com {
    reverse_proxy uku-app:8000
}

timer.towlion.com {
    reverse_proxy timer-app:8000
}

storage.towlion.com {
    reverse_proxy minio:9000
}
```

Caddy automatically provisions TLS certificates and redirects HTTP to HTTPS.

2.1.6 Database Strategy

A single PostgreSQL instance runs on the server. Each application uses a dedicated database for logical isolation:

```
uku_db
timer_db
lyrics_db
```

2.1.7 Persistent Storage

All persistent data is stored under `/data` on the host, ensuring data survives container redeployments.

```
/data
 ├── postgres          # PostgreSQL data
 ├── redis             # Redis data
 ├── minio             # Object storage
 ├── caddy/            # Caddy TLS certs and config
 │   ├── data
 │   └── config
 ├── loki              # Log aggregation data
 ├── grafana           # Dashboard data
 └── backups/postgres  # Database backups
```

Docker containers mount these directories as volumes.

2.1.8 Object Storage

MinIO provides S3-compatible object storage. Applications interact with it using the standard S3 API.

```
S3_ENDPOINT=https://storage.example.com
S3_BUCKET=uploads
S3_ACCESS_KEY=app_user
S3_SECRET_KEY=secret
```

Storage data lives at `/data/minio`.

2.1.9 Background Jobs

Asynchronous tasks are processed by Celery workers backed by Redis.

```
Application → Redis queue → Celery Worker
```

Typical use cases:

- Sending transactional email
- Processing file uploads
- Scheduled background tasks

Workers run as separate containers.

2.1.10 Transactional Email

Email is sent via external providers (e.g., Postmark, Amazon SES). The application sends email through provider APIs, with delivery handled by Celery workers.

```
EMAIL_PROVIDER=postmark
EMAIL_API_KEY=your-api-key
EMAIL_FROM=noreply@example.com
```

2.1.11 Logging and Backups

Logging: Applications write to stdout. Container logs are captured by the Docker runtime. Optionally, logs can be aggregated with Promtail, Loki, and Grafana.

Backups: Daily database backups via cron:

```
pg_dump → /data/backups
```

Backups can be synced to remote storage using `rclone`.

2.1.12 CI/CD Flow

```
graph LR
    Push["git push to main"] --> Actions["GitHub Actions"]

    subgraph Actions["GitHub Actions"]
        Test["Test Job"] --> Deploy["Deploy Job"]
    end

    Deploy -->|SSH| Server["Server"]

    subgraph Server["Server Operations"]
        Pull["git pull"] --> Build["docker compose up -d --build"]
        Build --> Trivy["Trivy image scan"]
        Build --> Migrate["Alembic migrate"]
        Migrate --> CaddyWrite["Write Caddyfile"]
        CaddyWrite --> Reload["Caddy reload"]
    end
```

2.1.13 Backup and Restore Flow

```
graph LR
    subgraph Backup["Daily Backup (cron 02:00)"]
        Cron["cron"] --> Script["backup-postgres.sh"]
        Script --> PgDump["pg_dump per database"]
        PgDump --> Compress["gzip"]
        Compress --> Store["/data/backups/postgres/"]
        Store --> Prune["Prune backups > 7 days"]
    end

    subgraph Restore["Manual Restore"]
        List["List backups"] --> Choose["Choose backup file"]
        Choose --> RestoreScript["restore-postgres.sh"]
        RestoreScript --> Drop["Drop + recreate DB"]
        Drop --> Import["Import from backup"]
        Import --> Verify["Verify data"]
    end
```

2.1.14 Preview Environment Flow

```
graph TB
    subgraph Create["PR Opened / Updated"]
        PR["Pull Request"] --> Actions["GitHub Actions"]
        Actions -->|SSH| Clone["Clone to /opt/apps/app-pr-N/"]
        Clone --> Schema["Create PR-specific DB schema"]
        Schema --> BuildPR["docker compose up -d --build"]
        BuildPR --> CaddyPR["Write pr-N.preview.app.domain.caddy"]
        CaddyPR --> ReloadPR["Caddy reload"]
        ReloadPR --> Comment["Comment preview URL on PR"]
    end

    subgraph Cleanup["PR Closed / Merged"]
        Closed["PR closed"] --> StopContainers["Stop + remove containers"]
        StopContainers --> DropSchema["Drop PR schema"]
```

```
        DropSchema --> RemoveCaddy["Remove .caddy file"]
        RemoveCaddy --> ReloadClean["Caddy reload"]
        ReloadClean --> RemoveDir["Remove /opt/apps/app-pr-N/"]
    end
```

2.1.15 Docker Compose Services

Each application lives in its own GitHub repository under the `towlion` organization. The server runs two layers of Compose services:

Platform Services (server-level)

Shared infrastructure managed at the server level, independent of any application repository:

```yaml
services:
  caddy:
    image: caddy:2
  postgres:
    image: postgres:16
    volumes:
      - /data/postgres:/var/lib/postgresql/data
  redis:
    image: redis
  minio:
    image: minio/minio
    volumes:
      - /data/minio:/data
```

Application Services (per-repo)

Each application repository defines its own containers. These connect to the shared platform services via Docker networking:

```yaml
services:
  app:
    build: ./app
    env_file: .env
  frontend:
    build: ./frontend
```

Celery workers are opt-in. To add background task processing, add a `celery-worker` service to your compose file. See the app-template README for instructions.

For single-app self-hosting (fork scenario), a repository may bundle platform services in its own Compose file so it can run standalone.

2.2 Application Specification

Spec Version: 1.0

This document defines the **standard contract for applications** running on the Towlion platform. Every application repository in the ecosystem should follow this specification.

2.2.1 Goals

All applications must be:

- Deployable automatically
- Self-hostable via repository forks
- Compatible with the platform runtime
- Easy to maintain

2.2.2 Repository Structure

Each application lives in its own GitHub repository under the `towlion` organization. Every application repository should follow this structure:

```
repo/
  app/                          # FastAPI backend
    __init__.py                 # Package marker (required)
    Dockerfile                  # Backend container image
    main.py                     # Application entry point
  frontend/                     # Next.js frontend (optional)
    Dockerfile                  # Frontend container image

  deploy/
    docker-compose.yml          # App-specific containers
    docker-compose.standalone.yml  # Full stack for self-hosted forks
    Caddyfile                   # Caddy site config for this app
    env.template

  .github/workflows/
    deploy.yml

  scripts/
    health-check.sh

  README.md
```

In the multi-app setup, `docker-compose.yml` defines only the application containers (app, frontend, workers). Shared platform services (Caddy, PostgreSQL, Redis, MinIO) are managed at the server level.

The `docker-compose.standalone.yml` file provides a complete, self-contained stack for self-hosted fork deployments. It bundles all platform services (Caddy, PostgreSQL, Redis, MinIO) alongside the application containers so the app can run on a single server without any external dependencies.

2.2.3 Backend Requirements

Applications must expose an HTTP server on **port 8000**.

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

The platform reverse proxy routes traffic to this port.

2.2.4 Health Endpoint

Every application must provide a health check endpoint:

- **Endpoint:** `GET /health`
- **HTTP Status:** `200 OK`
- **Response:** `{"status": "ok"}`

This endpoint is used during deployments to verify the application started correctly.

2.2.5 Environment Variables

Applications must support configuration via environment variables.

Required:

Variable	Purpose
`APP_DOMAIN`	Application domain
`DATABASE_URL`	PostgreSQL connection string
`REDIS_URL`	Redis connection string

Optional:

Variable	Purpose
`S3_ENDPOINT`	MinIO/S3 endpoint
`S3_BUCKET`	Storage bucket name
`S3_ACCESS_KEY`	Storage access key
`S3_SECRET_KEY`	Storage secret key
`JWT_SECRET`	Secret key for JWT token signing
`CORS_ORIGINS`	Comma-separated allowed CORS origins
`EMAIL_API_KEY`	Transactional email API key
`EMAIL_FROM`	Sender email address

2.2.6 Database

Applications should use PostgreSQL with Alembic for schema migrations.

Connection string format:

```
postgresql://user:password@postgres:5432/app_db
```

Migrations must run automatically during deployment.

2.2.7 Background Workers

Applications requiring async tasks should use Celery with Redis as the queue backend.

Typical use cases:

- Sending email
- Processing uploads
- Background processing

Workers run as separate containers.

2.2.8 Object Storage

File uploads should use S3-compatible object storage (MinIO).

```
S3_ENDPOINT=https://storage.example.com
S3_BUCKET=uploads
```

Applications should avoid storing large files on the container filesystem.

2.2.9 Frontend

Applications may include a frontend built with Next.js, React, and TypeScript. The frontend communicates with the backend via `/api`.

2.2.10 Dependencies

Python dependencies must be declared in one of:

- `requirements.txt` (default)
- `pyproject.toml` (alternative)

At minimum, dependencies must include `fastapi` and `uvicorn`.

2.2.11 Docker Requirements

Each repository must include Docker configuration. The backend Dockerfile must be located at `app/Dockerfile`. If the app includes a frontend, its Dockerfile must be at `frontend/Dockerfile`.

The container must:

- Start automatically
- Expose port 8000
- Read configuration from environment variables

Example `app/Dockerfile`:

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

2.2.12 Persistent Storage

Applications should not rely on container storage. Persistent data belongs in:

- Database (PostgreSQL)
- Object storage (MinIO)
- External services

2.2.13 Logging

Applications should write logs to stdout. Container logs are captured by the platform runtime.

2.2.14 Security

Applications should follow basic security practices:

- No hard-coded secrets
- Use environment variables for configuration
- Validate input data
- Enforce authentication where required

2.2.15 Authentication

Applications that require user authentication should use:

- **JWT tokens** (HS256) for session management
- **bcrypt** for password hashing
- **HTTPBearer** scheme for token transport

The app-template provides this infrastructure out of the box. See the app-template README for usage.

2.2.16 Compatibility Checklist

To remain compatible with the Towlion platform, applications must:

Structure: - [ ] Include `app/` directory with `Dockerfile` and `main.py` - [ ] Include `deploy/` directory with `docker-compose.yml`, `docker-compose.standalone.yml`, `Caddyfile`, and `env.template` - [ ] Include `.github/workflows/deploy.yml` - [ ] Include `scripts/health-check.sh` - [ ] Include `README.md`

Content: - [ ] `deploy/docker-compose.yml` is valid YAML, references port 8000, and includes a healthcheck - [ ] `deploy/env.template` contains `APP_DOMAIN`, `DATABASE_URL`, and `REDIS_URL` - [ ] `deploy/Caddyfile` contains `reverse_proxy` directive targeting port 8000 - [ ] `app/main.py` uses FastAPI - [ ] Python dependencies (`requirements.txt` or `pyproject.toml`) include `fastapi` and `uvicorn` - [ ] No hardcoded secrets in source code - [ ] `deploy/env.template` contains `JWT_SECRET` (if using authentication)

Runtime: - [ ] Expose HTTP service on port 8000 - [ ] `GET /health` returns HTTP 200 with `{"status": "ok"}` - [ ] Containers build and start successfully

2.2.17 Validation

Use the Towlion Spec Validator to automatically check conformance.

Local usage:

```
# Clone the platform repo and run against your app
python validator/validate.py --tier 2 --dir /path/to/your/app
```

CI usage (GitHub Actions):

```
steps:
  - uses: actions/checkout@v4
  - uses: towlion/platform/.github/actions/validate@main
    with:
      tier: '2'
```

2.3 Server Contract

This document defines the contract between the platform infrastructure (bootstrapped by `bootstrap-server.sh`) and the app deployment workflows (`deploy.yml`, `preview.yml`). It explains the directory layout, lifecycle, and assumptions that connect them.

2.3.1 Server Directory Layout

```
/opt/platform/                          # Platform root (created by bootstrap-server.sh)
  docker-compose.yml                    # Platform services (7 core + 3 optional metrics)
  .env                                  # Platform credentials (POSTGRES_PASSWORD, MINIO_ROOT_*, GRAFANA_ADMIN_PASSWORD, ACME_EMAIL, OPS_DOMAIN, ALERT_REPO,
COMPOSE_PROFILES)
  .bootstrapped                         # Timestamp marker from last bootstrap run
  Caddyfile                             # Global Caddyfile — imports /etc/caddy/apps/*.caddy
  caddy-apps/                           # Per-app Caddyfile fragments (written by deploy/preview workflows)
    <app-name>.caddy                    #   Production route for an app
    <app-name>-pr-<N>.caddy             #   Preview route for a PR
    ops.caddy                           #   Grafana route (created by bootstrap)
  credentials/                          # Per-app credential files (created by create-app-credentials.sh)
    <app-name>.env                      #   DB_USER, DB_PASSWORD, S3_ACCESS_KEY, S3_SECRET_KEY
  infrastructure/                       # Ops scripts (copied from platform repo by bootstrap)
    deploy-blue-green.sh                #   Blue-green zero-downtime deploy
    verify-backup.sh                    #   Backup restore verification
    rotate-credentials.sh               #   Credential rotation without downtime
  infrastructure/                       # Ops scripts (copied from platform repo by bootstrap)
    backup-postgres.sh
    restore-postgres.sh
    check-alerts.sh
    update-images.sh
    create-app-credentials.sh
    usage-report.sh
  loki-config.yml                       # Loki configuration
  promtail-config.yml                   # Promtail configuration
  prometheus.yml                        # Prometheus scrape config (always created)
  grafana/                              # Grafana provisioning files and dashboards
    provisioning/datasources/
    provisioning/dashboards/
    dashboards/

/opt/apps/                              # Application root (created by bootstrap-server.sh)
  <app-name>/                           # Cloned app repo (created manually by admin)
    .deploy-slot                        # Active blue-green slot ("blue" or "green")
    deploy/
      .env                              # App runtime config (created manually from env.template)
      docker-compose.yml                # App services (joins towlion network)
  <app-name>-pr-<N>/                    # Preview clone (created/destroyed by preview.yml)

/data/                                  # Persistent data volumes (created by bootstrap-server.sh)
  postgres/                             # PostgreSQL data directory
  redis/                                # Redis data directory
  minio/                                # MinIO object storage
  caddy/data/                           # Caddy TLS certificates
  caddy/config/                         # Caddy config state
  loki/                                 # Loki log storage
  grafana/                              # Grafana state
  prometheus/                           # Prometheus data (created always, used when metrics enabled)
  backups/postgres/                     # pg_dump backup files (7-day retention, optional .dump.enc encryption)
```

2.3.2 Bootstrap to Deploy Lifecycle

1. **Bootstrap the server** — Run `sudo bash infrastructure/bootstrap-server.sh` on a fresh Debian machine. This creates the directory layout above, installs Docker, creates the `deploy` user, generates platform credentials, starts the 7 core platform services (plus 3 optional metrics services if enabled), copies infrastructure scripts, and installs cron jobs.
2. **Configure DNS** — Point app domains and `*.preview.<app>.<domain>` (one per app) to the server IP.
3. **Clone the app repo** — SSH in as `deploy` and clone the app to `/opt/apps/<name>/`.
4. **Create `deploy/.env`** — Copy `deploy/env.template` and fill in values (DATABASE_URL, S3 credentials, etc.).
5. **Provision per-app credentials (optional)** — Run `create-app-credentials.sh <name>` to create an isolated PostgreSQL user and MinIO bucket. Credentials are written to `/opt/platform/credentials/<name>.env`.
6. **Configure GitHub secrets** — Set `SERVER_HOST`, `SERVER_USER`, `SERVER_SSH_KEY`, and `APP_DOMAIN` on the app repo. Add `PREVIEW_DOMAIN` for preview environments.
7. **Push to main** — Triggers `deploy.yml`:

8. SSHes into the server as `deploy`
9. Calls `deploy-blue-green.sh` which performs a zero-downtime blue-green deploy:
   - Reads current slot from `/opt/apps/<name>/.deploy-slot` (defaults to "blue")
   - Pulls latest code, injects credentials, builds and starts the next slot
   - Waits for Docker healthcheck, runs Alembic migrations
   - Swaps Caddyfile to point to the new slot, reloads Caddy
   - Verifies external health, then stops the old slot
   - On any failure: tears down the new slot, old slot keeps serving traffic

2.3.3 App Workflow Server Assumptions

The `deploy.yml` and `preview.yml` workflows SSH into the server and depend on the following structure being in place:

Path / Resource	Purpose	Created By
`/opt/platform/docker-compose.yml`	Platform services (postgres, redis, caddy, etc.)	`bootstrap-server.sh`
`/opt/platform/.env`	`POSTGRES_PASSWORD` for database operations	`bootstrap-server.sh`
`/opt/platform/caddy-apps/`	Writable directory for per-app Caddyfile fragments	`bootstrap-server.sh`
`/opt/platform/credentials/<name>.env`	Per-app DB/S3 credentials (optional)	`create-app-credentials.sh`
`/opt/apps/<name>/`	Cloned app repo with `deploy/.env` configured	Admin (manual)
`towlion` Docker network	Shared network connecting platform services and app containers	`bootstrap-server.sh`
`deploy` user	SSH user with Docker group membership	`bootstrap-server.sh`

If any of these are missing, the workflow will fail. The bootstrap script is idempotent and can be re-run to restore missing structure.

2.3.4 Infrastructure Scripts Reference

All scripts live in the platform repo under `infrastructure/` and are copied to `/opt/platform/infrastructure/` during bootstrap.

Script	Purpose	Invocation
`bootstrap-server.sh`	Transform fresh Debian into running platform	Manual (`sudo bash`)
`verify-server.sh`	Read-only health check of server state	Manual (`bash`)
`create-app-credentials.sh`	Provision per-app PostgreSQL user + MinIO bucket	Manual (`bash <script> <app-name>`)
`backup-postgres.sh`	Per-database `pg_dump` with 7-day retention, optional AES-256 encryption	Cron: daily at 02:00
`restore-postgres.sh`	Restore a database from backup	Manual (`bash <script>`)
`check-alerts.sh`	Check container health, disk, memory; create GitHub Issues	Cron: every 5 minutes
`update-images.sh`	Pull latest Docker images and recreate containers	Cron: weekly Sunday at 03:00
`usage-report.sh`	Generate 6-section resource usage report	Manual (`bash`)
`scan-images.sh`	Scan running container images for vulnerabilities (Trivy), create GitHub Issues	Cron: weekly Sunday at 04:00
`deploy-blue-green.sh`	Zero-downtime blue-green deploy with automatic rollback	Called by `deploy.yml` workflow
`verify-backup.sh`	Restore backups to temp DB and verify integrity	Cron: weekly Sunday at 05:00
`rotate-credentials.sh`	Rotate per-app or platform master credentials without downtime	Manual (`bash <script> <app-name>` or `--platform`)

2.3.5 Server Hardening

The bootstrap script applies several security measures automatically. Self-hosters get these out of the box; no manual hardening steps are needed beyond running the bootstrap.

Firewall — UFW is configured to deny all incoming traffic except ports 22 (SSH), 80 (HTTP), and 443 (HTTPS). All other ports, including PostgreSQL (5432), Redis (6379), and MinIO (9000), are only reachable via the internal Docker network.

Automatic security updates — `unattended-upgrades` is installed and configured for the Debian security channel. Security patches are applied automatically without manual intervention.

Non-root deployment — A `deploy` user with Docker group membership handles all deployments. Workflows SSH in as `deploy`, never as root. The deploy user cannot modify system packages or firewall rules.

Credential isolation — The platform `.env` file is mode 600 (readable only by its owner). Per-app credentials are generated by `create-app-credentials.sh` and stored in separate files under `/opt/platform/credentials/`, each also mode 600.

Container resource limits — Every platform service and every app container has explicit CPU and memory limits in its Docker Compose file. This prevents any single container from exhausting server resources. The 7 core services use ~2.66G / 3.25 CPU. The 3 optional metrics services add 448M / 1.00 CPU when enabled.

Brute-force protection — fail2ban is installed with an SSH jail (systemd backend, since Debian 12 uses journald). Configuration: maxretry=5, bantime=3600s. IPs that exceed the retry limit are banned for one hour.

SSH hardening — A drop-in config at `/etc/ssh/sshd_config.d/99-towlion-hardening.conf` enforces: `PermitRootLogin no`, `PasswordAuthentication no`, `MaxAuthTries 3`, `X11Forwarding no`. Only key-based authentication as the `deploy` user is permitted.

Security headers — The platform Caddyfile includes a `(security_headers)` snippet that sets: `Strict-Transport-Security` (HSTS, max-age=31536000, includeSubDomains), `X-Content-Type-Options nosniff`, `X-Frame-Options DENY`, `Referrer-Policy strict-origin-when-cross-origin`, `Permissions-Policy` (camera, microphone, and geolocation denied), and strips the `Server` header. All app and ops Caddy routes import this snippet.

Rate limiting — Application-level rate limiting via `slowapi` (FastAPI middleware). Default limit: 60 requests/minute per IP. The `/health` endpoint is exempt from rate limiting since Docker healthchecks and monitoring hit it frequently.

Read-only container filesystems — App containers, Caddy, and Promtail run with `read_only: true` in their Docker Compose configuration. Writable areas (`/tmp`, `/app/__pycache__`) are mounted as `tmpfs`. Database services (PostgreSQL, Redis, MinIO) are not read-only due to PID files and temp tables.

Docker event audit logging — A systemd service (`docker-audit.service`) runs `docker events` with JSON output to `/var/log/docker-audit.log`. Promtail scrapes this file and forwards events to Loki (label: `job=docker-audit`). All container start, stop, die, and health_status events are captured.

Backup encryption — Backups can be encrypted at rest using AES-256-CBC. Set the `BACKUP_ENCRYPTION_KEY` environment variable to the path of a key file. When set, `backup-postgres.sh` pipes `pg_dump` output through `openssl enc` and produces `.dump.enc` files. `restore-postgres.sh` and `verify-backup.sh` automatically detect encrypted backups and decrypt them before restoring. If the key file is not set, backups are stored unencrypted (with a warning).

Log rotation — A logrotate config at `/etc/logrotate.d/towlion` rotates `/var/log/towlion-*.log` and `/var/log/docker-audit.log` daily, retaining 90 compressed copies. The `docker-audit.service` is restarted after rotation since it holds the log file open.

Log retention — Loki retains logs for 90 days (`retention_period: 2160h`). The compactor runs retention enforcement with a 2-hour delete delay.

Platform credential rotation — `rotate-credentials.sh --platform` rotates the PostgreSQL superuser password and/or MinIO root password. After rotation, all app health checks are verified. Use `--yes` to skip the confirmation prompt.

Image vulnerability scanning — Trivy is installed via the Aqua Security apt repository. Every deploy runs a non-blocking `trivy image` scan of the newly built app image (HIGH/CRITICAL severity). A weekly cron job (`scan-images.sh`, Sunday 04:00) scans all running container images.

Mandatory Access Control (AppArmor) — Debian 12 ships with AppArmor enabled by default. Docker automatically applies the `docker-default` AppArmor profile to all containers, which restricts capabilities like writing to `/proc` and `/sys`, mounting filesystems, and accessing raw sockets. No configuration is needed — this works out of the box.

SELinux is **not** used. While SELinux is the standard MAC system on RHEL/Fedora, it is not well-suited for Debian:

- AppArmor is Debian's native MAC system, maintained by the Debian security team
- SELinux policies on Debian are incomplete and poorly maintained — the `selinux-policy-default` package lags far behind RHEL equivalents
- Docker + SELinux on Debian causes bind-mount labeling issues (`:z`/`:Z` volume flags) with no community support for troubleshooting
- Enabling SELinux on Debian requires switching from AppArmor, losing Docker's automatic profile enforcement

Since AppArmor is already active and Docker integrates with it automatically, the platform's MAC requirements are met without any additional configuration.

2.3.6 Caddyfile Generation

The platform Caddyfile at `/opt/platform/Caddyfile` contains a security headers snippet and an import directive:

```
{
    email {$ACME_EMAIL:admin@localhost}
}

(security_headers) {
    header {
        Strict-Transport-Security "max-age=31536000; includeSubDomains"
        X-Content-Type-Options "nosniff"
        X-Frame-Options "DENY"
```

```
        Referrer-Policy "strict-origin-when-cross-origin"
        Permissions-Policy "camera=(), microphone=(), geolocation=()"
        -Server
    }
}

import /etc/caddy/apps/*.caddy
```

The `caddy-apps/` directory is bind-mounted into the Caddy container at `/etc/caddy/apps/`. App workflows write per-app `.caddy` files into this directory.

Production (`deploy.yml`) writes `/opt/platform/caddy-apps/<name>.caddy` via `deploy-blue-green.sh`. Container names include the active slot:

```
app.example.com {
    import security_headers
    reverse_proxy <name>-<slot>-app-1:8000
}
```

Preview (`preview.yml`) writes `/opt/platform/caddy-apps/<name>-pr-<N>.caddy`:

```
pr-<N>.preview.<name>.example.com {
    import security_headers
    reverse_proxy <name>-pr-<N>-app-1:8000
}
```

After writing the file, both workflows reload Caddy:

```
docker compose -f /opt/platform/docker-compose.yml exec -T caddy caddy reload --config /etc/caddy/Caddyfile
```

Preview cleanup removes the `.caddy` file and reloads Caddy again.

2.3.7 Per-App Credentials

By default, apps connect to PostgreSQL as the `postgres` superuser (credentials from `deploy/.env`). For credential isolation, run:

```
bash /opt/platform/infrastructure/create-app-credentials.sh <app-name>
```

This creates:

- **PostgreSQL**: A dedicated user (`<app_name>_user`) with access restricted to `<app_name>_db`
- **MinIO**: A dedicated user (`<app-name>-user`) with a scoped policy limiting access to the `<app-name>-uploads` bucket
- **Credentials file**: `/opt/platform/credentials/<app-name>.env` containing `DB_USER`, `DB_PASSWORD`, `S3_ACCESS_KEY`, `S3_SECRET_KEY` (mode 600, owned by `deploy`)

On subsequent deploys, `deploy.yml` checks for this credentials file and, if found, updates `deploy/.env` with the per-app values via `sed`:

```
CREDENTIALS_FILE="/opt/platform/credentials/${APP_NAME}.env"
if [ -f "$CREDENTIALS_FILE" ]; then
    source "$CREDENTIALS_FILE"
    sed -i "s|^DATABASE_URL=.*|DATABASE_URL=postgresql://${DB_USER}:${DB_PASSWORD}@postgres:5432/${APP_DB}|" deploy/.env
    sed -i "s|^S3_ACCESS_KEY=.*|S3_ACCESS_KEY=${S3_ACCESS_KEY}|" deploy/.env
    sed -i "s|^S3_SECRET_KEY=.*|S3_SECRET_KEY=${S3_SECRET_KEY}|" deploy/.env
    sed -i "s|^S3_BUCKET=.*|S3_BUCKET=${APP_NAME}-uploads|" deploy/.env
fi
```

If no credentials file exists, the workflow falls back to whatever is already in `deploy/.env` and logs a warning.

2.3.8 Resource Metrics (Optional)

Three additional services provide real-time resource visibility in Grafana. They are **off by default** and use Docker Compose profiles to control startup.

Service	Image	Memory Limit	CPU Limit	Purpose
prometheus	prom/prometheus:v2.53.0	256M	0.50	Metrics storage and queries
cadvisor	gcr.io/cadvisor/cadvisor:v0.49.1	128M	0.25	Container metrics
node-exporter	prom/node-exporter:v1.8.1	64M	0.25	Host metrics

To enable — add `COMPOSE_PROFILES=metrics` to `/opt/platform/.env`, then run `docker compose up -d` in `/opt/platform/`. Or bootstrap with `sudo ENABLE_METRICS=true bash bootstrap-server.sh`.

To disable — remove the `COMPOSE_PROFILES=metrics` line from `.env`, then stop the metrics services with `docker compose --profile metrics down`.

The Prometheus config (`/opt/platform/prometheus.yml`), Grafana datasource, and dashboard JSON are always created during bootstrap — they are harmless without running services and avoid needing to re-bootstrap to enable metrics later. Only the service startup is conditional.

The "Resource Metrics" dashboard in Grafana includes:

- **Host overview**: CPU %, memory %, disk %, uptime (stat panels)
- **Host time series**: CPU, memory, disk I/O, network I/O over time
- **Container overview**: table of all containers with CPU, memory, network
- **Per-container detail**: filterable CPU and memory time series per container

3. Deployment

3.1 Deployment

3.1.1 Deployment Pipeline

Applications are deployed automatically through GitHub Actions. The full pipeline:

```
Developer Push
       │
       ▼
GitHub Actions
       │
       ├── Run Tests
       ├── Build Docker Image
       ├── SSH into Server
       ├── Deploy Containers
       ├── Run Database Migrations (inside container)
       ├── Health Check
       └── Enable Traffic
```

3.1.2 GitHub Actions Workflow

Every repository includes a deployment workflow at `.github/workflows/deploy.yml`.

Typical steps:

1. Checkout repository
2. Run tests
3. Build Docker image
4. SSH into server
5. Pull latest code
6. Restart containers
7. Run health checks

Deployment command:

```
docker compose up -d --build
```

3.1.3 Deploy Behavior

The platform rebuilds and restarts containers on each deploy. There is a brief gap where the application is unavailable.

```
Stop current container
        │
        ▼
Rebuild image from source
        │
        ▼
Start new container
        │
        ▼
Health check passes
        │
        ▼
Caddy routes traffic
```

During the rebuild and restart, Caddy returns 502 for requests to the application. This gap is typically a few seconds. For the target use case — small apps with low traffic — this is acceptable. See Scope and Design Boundaries for when to consider alternatives.

Docker Health Check Configuration

```yaml
services:
  app:
    build: ./app
    restart: always
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 10s
      timeout: 5s
      retries: 5
```

If the health check fails, Docker will restart the container according to the `restart: always` policy.

3.1.4 Database Migrations

Migrations run automatically as part of the deployment pipeline, **after the application container starts, by executing alembic inside the running container**.

```
Deploy started
      │
      ▼
Start containers
      │
      ▼
Run migrations (inside container)
      │
      ▼
Health check
```

Migration command:

```
docker compose exec app alembic -c app/alembic.ini upgrade head
```

Safe Migration Rules

To avoid downtime during schema changes:

1. **Add** columns before removing columns
2. Avoid destructive migrations
3. Use multi-step schema evolution

Example safe pattern:

```
Deploy 1: Add new column
Deploy 2: Start using new column
Deploy 3: Remove old column
```

3.1.5 Health Checks

Every application must provide a health endpoint:

```
GET /health → {"status": "ok"}
```

After deployment, the workflow verifies the application is healthy:

```
curl https://app.example.com/health
```

3.1.6 Environment Promotion

The platform has two environments:

```
Pull Request → Preview environment
Merge to main → Production deployment
```

Branch	Environment
PR branch	Preview
`main`	Production

Preview environments are created automatically for pull requests and cleaned up when the PR is closed. See Preview Environments for details.

3.1.7 Platform Capabilities Summary

Feature	Implementation
CI/CD	GitHub Actions
Deploy pipeline	Rebuild and restart on push to main
Database migrations	Alembic
Preview environments	PR deployments
Object storage	MinIO
Queue system	Redis + Celery
Reverse proxy	Caddy
TLS	Automatic via Let's Encrypt
Persistent storage	`/data` volume
Transactional email	External provider

3.2 Self-Hosting

3.2.1 Fork-Based Deployment Model

Every application in the Towlion ecosystem is designed to be **forkable and self-deployable**. Instead of multi-tenant SaaS, the platform uses repository forks as the isolation boundary.

```
User forks repository
      ↓
Configure deployment secrets
      ↓
Push code
      ↓
GitHub Actions deploys app
      ↓
Application runs on user's server
```

Each fork runs independently — there is no shared tenancy or centralized hosting.

```
original repo
     │
     ├─ fork by Alice → alice.example.com
     ├─ fork by Bob   → bob.example.com
     └─ fork by Carol → carol.example.com
```

Benefits:

- Strong isolation between deployments
- Simple architecture (no tenant management)
- Full control for each operator

3.2.2 Server Requirements

Resource	Minimum
CPU	2 cores
RAM	4 GB
Disk	50 GB
Data disk	Mounted at `/data`

Required ports:

Port	Purpose
22	SSH
80	HTTP
443	HTTPS

3.2.3 Required Secrets

Fork users must configure these GitHub Actions secrets:

Secret	Purpose
`SERVER_HOST`	Server IP address
`SERVER_USER`	SSH user (typically `deploy`)
`SERVER_SSH_KEY`	SSH private key for deployment
`APP_DOMAIN`	Application domain name (e.g., `app.example.com`)

Note: Database and storage credentials are auto-generated by the bootstrap script on the server. They are not GitHub secrets.

Optional secrets:

Secret	Purpose
`PREVIEW_DOMAIN`	Base domain for preview environments (e.g., `example.com`)

Tip: To reduce the number of required secrets from 4 to 1, you can use a self-hosted GitHub Actions runner instead. See Self-Hosted Runners.

3.2.4 Bootstrap Process

To deploy an application from a fork:

1. **Fork the repository** on GitHub
2. **Create a server** (Debian, meeting the minimum requirements)
3. **Run the bootstrap script** to install Docker and dependencies
4. **Configure DNS** — point your domain to the server IP
5. **Configure GitHub secrets** — add the required secrets to your fork
6. **Push code** — deployment runs automatically via GitHub Actions

Bootstrap Script

The repository includes a single bootstrap script that transforms a fresh Debian server into a ready-to-deploy platform:

```
sudo ACME_EMAIL=you@example.com OPS_DOMAIN=ops.example.com bash infrastructure/bootstrap-server.sh
```

The script is idempotent — safe to re-run on an already-bootstrapped server. It performs:

- System packages (git, curl, ufw, vnstat, unattended-upgrades)
- Firewall configuration (ports 22, 80, 443)
- Docker and Compose plugin installation
- `deploy` user creation with SSH directory
- Directory structure (`/data/*`, `/opt/apps`, `/opt/platform`)
- Docker network (`towlion`) for cross-container communication
- Credential generation (PostgreSQL, MinIO, and Grafana passwords in `/opt/platform/.env`)
- Platform Caddyfile with per-app import pattern
- Platform `docker-compose.yml` (PostgreSQL 16, Redis 7, MinIO, Caddy 2, Loki 3.0, Promtail 3.0, Grafana 11.0)
- Service startup and verification
- Infrastructure scripts and cron jobs (backups, alerts, image updates)

Post-Bootstrap Verification

After bootstrapping, run the verification script to confirm everything is working:

```
bash infrastructure/verify-server.sh
```

This performs a read-only check of all services, directories, and firewall rules.

Per-App Credentials

For each application, provision isolated database and storage credentials:

```
bash /opt/platform/infrastructure/create-app-credentials.sh <app-name>
```

This creates a dedicated PostgreSQL user and MinIO bucket. Credentials are written to `/opt/platform/credentials/<app-name>.env` and automatically picked up by the deploy workflow.

Monitoring (Optional)

If you set `OPS_DOMAIN` during bootstrap (or add it to `/opt/platform/.env`), Grafana is accessible at `https://OPS_DOMAIN`. The default admin credentials are in `/opt/platform/.env`. Configure DNS for the ops domain the same way as app domains.

3.2.5 DNS Configuration

Point your domain (or subdomain) to your server's IP address:

```
A record: app.example.com → SERVER_IP
```

For multiple applications, use subdomains:

```
uku.example.com   → SERVER_IP
timer.example.com → SERVER_IP
```

Caddy handles TLS certificate provisioning automatically once DNS is configured.

3.2.6 What You Get

After deployment, your server runs:

- Your application (FastAPI backend + Next.js frontend)
- PostgreSQL database
- Redis cache and queue
- MinIO object storage
- Caddy reverse proxy with automatic TLS
- Celery background workers

All managed through GitHub — push code to deploy updates.

For the full directory layout, lifecycle sequence, and workflow assumptions, see Server Contract.

3.3 Preview Environments

Preview environments allow developers to see changes **before merging into main**. Each pull request gets a temporary deployment with its own URL.

3.3.1 How It Works

```
Pull Request opened/updated
        |
        v
GitHub Actions (preview.yml) triggered
        |
        v
Deploy preview container + DB schema
        |
        v
pr-42.preview.todo-app.anulectra.com
```

Each preview environment gets:

- Separate application container (project name `{APP_NAME}-pr-{N}`)
- Isolated database schema (`pr_{N}` in the app's existing database)
- Independent Caddyfile route
- Automatic PR comment with preview URL

3.3.2 DNS Configuration

Each app needs a per-app wildcard DNS A record pointing preview subdomains to the platform server:

```
*.preview.todo-app.anulectra.com → 143.198.104.8  (TTL 3600)
*.preview.wit.anulectra.com      → 143.198.104.8  (TTL 3600)
```

This allows any preview subdomain to resolve automatically:

```
pr-1.preview.todo-app.anulectra.com
pr-42.preview.todo-app.anulectra.com
pr-5.preview.wit.anulectra.com
```

3.3.3 GitHub Secrets

Apps need one additional secret beyond the standard 4:

Secret	Example	Purpose
`PREVIEW_DOMAIN`	`anulectra.com`	Base domain for preview URLs

The preview URL is constructed as `pr-{N}.preview.{APP_NAME}.{PREVIEW_DOMAIN}`.

3.3.4 Deployment Workflow

The preview workflow (`.github/workflows/preview.yml`) triggers on pull request events:

```yaml
on:
  pull_request:
    types: [opened, synchronize, reopened, closed]
```

Deploy (opened/synchronize/reopened)

1. SSH to server, clone repo to `/opt/apps/{APP_NAME}-pr-{N}` (or pull if it already exists)

2. Checkout the PR branch in the dedicated preview directory
3. Create PostgreSQL schema `pr_{N}` in the app's database
4. Copy production `.env` from `/opt/apps/{APP_NAME}/deploy/.env` and generate `.env.pr-{N}` with schema-aware `DATABASE_URL`
5. Build and start containers: `docker compose -p {APP_NAME}-pr-{N} -f deploy/docker-compose.yml up -d --build`
6. Run Alembic migrations against the preview schema
7. Write Caddyfile to `/opt/platform/caddy-apps/{APP_NAME}-pr-{N}.caddy`
8. Reload Caddy
9. Post/update PR comment with preview URL

Cleanup (closed)

1. Stop and remove containers: `docker compose -p {APP_NAME}-pr-{N} down --rmi local`
2. Drop schema: `DROP SCHEMA IF EXISTS pr_{N} CASCADE`
3. Remove Caddyfile: `rm /opt/platform/caddy-apps/{APP_NAME}-pr-{N}.caddy`
4. Reload Caddy
5. Remove the preview directory: `rm -rf /opt/apps/{APP_NAME}-pr-{N}`

3.3.5 Database Schema Isolation

Preview environments use PostgreSQL schemas rather than separate databases:

- **Production**: uses `{app}_db` database, default `public` schema
- **Preview PR 42**: uses `{app}_db` database, schema `pr_42`

The schema-aware `DATABASE_URL` uses the `options` parameter:

```
postgresql://postgres:<password>@postgres:5432/todo_app_db?options=-csearch_path%3Dpr_42
```

Alembic migrations run inside the preview container against the PR schema automatically.

3.3.6 Container Naming

Preview containers use the Docker Compose project name `{APP_NAME}-pr-{N}`:

```
todo-app-pr-42-app-1
todo-app-pr-42-celery-worker-1
```

3.3.7 Caddy Routing

Each preview gets a Caddyfile at `/opt/platform/caddy-apps/{APP_NAME}-pr-{N}.caddy`:

```
pr-42.preview.todo-app.anulectra.com {
    reverse_proxy todo-app-pr-42-app-1:8000
}
```

This uses the existing `import /opt/platform/caddy-apps/*.caddy` pattern — no platform Caddy config changes are needed.

3.3.8 Concurrent Previews

Multiple PRs can be previewed simultaneously. Each gets its own: - Container set (unique project name) - Database schema (unique schema name) - Caddy route (unique Caddyfile)

There is no limit beyond server resources.

3.4 Deploy Prompt Template

A reusable Claude Code session prompt for spinning up a new Towlion application deployment. Copy this prompt, replace the placeholders, and paste it into a Claude Code session.

3.4.1 Placeholders

Placeholder	Description	Example
`<app-name>`	Repository name for the new app	`todo-app`
`<your-domain.com>`	Domain or subdomain for the app	`todo.example.com`
`<server-ip>`	IP address of the target server	`203.0.113.42`

3.4.2 Prompt

```
I want to deploy a new Towlion app called <app-name> to <your-domain.com> on server <server-ip>.

Walk me through these 5 phases:

### Phase 1: Create repository
- Create a new repo from the towlion/app-template GitHub template
- Clone it locally
- Run setup-repo.sh to configure governance (branch protection, labels)

### Phase 2: Build the app
- Implement the application logic in app/main.py
- Add any new dependencies to requirements.txt (at repo root)
- Ensure the /health endpoint returns {"status": "ok"}
- Commit with conventional commit messages (feat:, fix:, etc.)

### Phase 3: Provision server
- Run `bootstrap-server.sh` on a fresh Debian server (installs Docker, creates directories, starts platform services)
- Set up DNS A record: <your-domain.com> -> <server-ip>
- Generate a deploy SSH key pair for the `deploy` user
- Run `verify-server.sh` to confirm the platform is healthy

### Phase 4: Deploy
- SSH to server: create /opt/apps/<app-name>, clone the repo, create deploy/.env from deploy/env.template using credentials from /opt/platform/.env
- Configure GitHub Actions secrets (SERVER_HOST, SERVER_USER, SERVER_SSH_KEY, APP_DOMAIN)
- Push to main to trigger the deploy workflow
- The workflow will: verify .env exists, auto-create the app database, build containers, run migrations, health check, generate Caddyfile, reload Caddy
- Verify with: curl https://<your-domain.com>/health

### Phase 5: Configure CI/CD
- Confirm the deploy.yml workflow runs on push to main
- Confirm the validate.yml workflow runs on PRs
- Test by pushing a small change and watching the Actions tab

Important notes:
- The app-template already includes all deployment fixes (correct migration ordering, __init__.py, PYTHONPATH, alembic -c flag)
- Multi-app mode uses deploy/docker-compose.yml which joins the shared `towlion` network (platform provides postgres, redis, minio, caddy)
- The deploy workflow auto-creates a per-app database (e.g., todo_app_db) — no manual DB setup needed
- Caddyfile is generated by the workflow (not symlinked) with the literal domain and project-prefixed container name
- deploy/.env must exist on the server before the first push — the workflow fails fast if missing
- requirements.txt must be at repo root (not app/) for the Dockerfile build context
```

3.4.3 See also

- Tutorial — step-by-step walkthrough with detailed commands
- Deployment — pipeline internals and deploy behavior
- Self-Hosting — fork model and infrastructure overview

3.5 Self-Hosted GitHub Actions Runners

3.5.1 Overview

By default, Towlion deploys applications using **GitHub-hosted runners** that SSH into your server. An alternative approach is to run a **self-hosted GitHub Actions runner** directly on your server. This eliminates the SSH dependency entirely — the runner executes deploy commands locally instead of over a network connection.

3.5.2 Comparison

	GitHub-Hosted (default)	Self-Hosted Runner
Required secrets	4 (`SERVER_HOST`, `SERVER_USER`, `SERVER_SSH_KEY`, `APP_DOMAIN`)	1 (`APP_DOMAIN`)
Network dependency	SSH from GitHub to your server	Runner polls GitHub for jobs
Deploy speed	Slower (SSH overhead, file transfer)	Faster (commands run locally)
Maintenance	None — GitHub manages the runner	You maintain the runner binary and keep it updated
Security model	SSH key grants deploy access	Runner process has local access; compromise = server compromise
Firewall	Port 22 must be open	No inbound ports required (runner uses outbound HTTPS)

3.5.3 When to Use Self-Hosted Runners

Consider self-hosted runners if:

- You want to **reduce secrets** from 4 to 1
- You want **faster deploys** without SSH overhead
- You prefer **no inbound SSH** on your server (tighter firewall)
- You're comfortable maintaining the runner software

Stick with GitHub-hosted runners (the default) if:

- You prefer **zero maintenance** on the runner side
- You want the **simplest setup** with no extra software
- You don't want to manage runner updates or monitor runner health

3.5.4 Setup

1. Install the Runner

On your server, as the `deploy` user:

```
# Create a directory for the runner
mkdir -p ~/actions-runner && cd ~/actions-runner

# Download the latest runner (check https://github.com/actions/runner/releases for current version)
# NOTE: The version below (v2.321.0) is an example — replace with the current release from the link above.
curl -o actions-runner-linux-x64.tar.gz -L \
  https://github.com/actions/runner/releases/download/v2.321.0/actions-runner-linux-x64-2.321.0.tar.gz

tar xzf actions-runner-linux-x64.tar.gz
```

2. Register the Runner

Go to your fork's **Settings > Actions > Runners > New self-hosted runner** and copy the registration token. Then:

```
./config.sh --url https://github.com/YOUR_USER/YOUR_REPO --token YOUR_TOKEN
```

Accept the defaults or customize the runner name and labels.

3. Install as a Service

```
sudo ./svc.sh install deploy
sudo ./svc.sh start
```

This creates a systemd service that starts the runner on boot and keeps it running.

4. Verify

Check that the runner appears as "Idle" in your fork's **Settings > Actions > Runners**.

3.5.5 Workflow Changes

With a self-hosted runner, your deploy workflow no longer needs SSH. Here's what to change:

deploy.yml

Replace:

```yaml
runs-on: ubuntu-latest
```

With:

```yaml
runs-on: self-hosted
```

Remove the SSH action and replace remote commands with local commands. For example, instead of:

```yaml
- name: Deploy via SSH
  uses: appleboy/ssh-action@v1
  with:
    host: ${{ secrets.SERVER_HOST }}
    username: ${{ secrets.SERVER_USER }}
    key: ${{ secrets.SERVER_SSH_KEY }}
    script: |
      cd /opt/apps/$APP_NAME
      docker compose -f deploy/docker-compose.yml pull
      docker compose -f deploy/docker-compose.yml up -d
```

Use:

```yaml
- name: Deploy
  run: |
    cd /opt/apps/$APP_NAME
    git pull origin main
    docker compose -f deploy/docker-compose.yml pull
    docker compose -f deploy/docker-compose.yml up -d
```

The same pattern applies to `preview.yml` — replace `runs-on` and convert SSH commands to local commands.

Secrets

With self-hosted runners, you only need:

Secret	Purpose
`APP_DOMAIN`	Application domain name

The other 3 secrets (`SERVER_HOST`, `SERVER_USER`, `SERVER_SSH_KEY`) are no longer needed since commands run directly on the server.

3.5.6 Security Considerations

- **Runner compromise = server compromise.** The runner process can execute arbitrary commands as the `deploy` user. Protect your repository — anyone with push access can run code on your server via workflow files.
- **Keep the runner updated.** GitHub periodically releases security patches. The runner auto-updates by default, but verify this is working.
- **Restrict fork PRs.** By default, workflows from fork pull requests don't run on self-hosted runners. Keep this default — allowing fork PRs to trigger workflows on your server is a significant security risk.
- **Use a dedicated user.** The runner should run as the `deploy` user (not root), which is already the case if you follow the setup above.

3.5.7 Troubleshooting

Runner shows as "Offline"

Check the service status:

```
sudo ./svc.sh status
```

If stopped, restart it:

```
sudo ./svc.sh start
```

Jobs stay queued

Verify the runner labels match the `runs-on` value in your workflow. The default label is `self-hosted`. Check with:

```
./config.sh --list
```

Runner won't update

The runner normally auto-updates. If it fails, manually download the latest release and re-extract:

```
cd ~/actions-runner
sudo ./svc.sh stop
curl -o actions-runner-linux-x64.tar.gz -L <latest-release-url>
tar xzf actions-runner-linux-x64.tar.gz
sudo ./svc.sh start
```

4. Operations

4.1 Towlion CLI

The `towlion` CLI wraps common platform operations into a single command. It uses SSH for server operations and the GitHub CLI (`gh`) for workflow triggers.

4.1.1 Installation

```
# From the platform repo
cp cli/towlion /usr/local/bin/towlion
chmod +x /usr/local/bin/towlion
```

4.1.2 Configuration

Create `~/.towlion.conf`:

```
SERVER_HOST=143.198.104.8
SERVER_USER=deploy
SSH_KEY_PATH=~/.ssh/id_rsa
```

4.1.3 Profiles

The config file supports INI-style `[profile]` sections for managing multiple servers. Existing flat configs (no section headers) are treated as `[default]`.

```
[default]
SERVER_HOST=143.198.104.8
SERVER_USER=deploy
SSH_KEY_PATH=~/.ssh/id_rsa

[staging]
SERVER_HOST=10.0.0.5
SERVER_USER=deploy
SSH_KEY_PATH=~/.ssh/staging_key
```

Select a profile with `--profile` or the `TOWLION_PROFILE` environment variable:

```
towlion --profile staging status
TOWLION_PROFILE=staging towlion status
```

If no profile is specified, `default` is used.

4.1.4 Commands

`towlion status`

List all apps with container state and health status.

`towlion logs <app>`

Tail logs for an app (uses `docker compose logs -f`).

`towlion health [app|all]`

Check `/health` endpoints. Defaults to `all` if no app specified.

`towlion create <app>`

Create a new app from the template: 1. Creates GitHub repo from `towlion/app-template` 2. Provisions PostgreSQL and MinIO credentials on the server 3. Clones the repo on the server 4. Creates `deploy/.env` from template

Requires `gh` CLI to be installed and authenticated.

`towlion secrets <app> <domain> [--preview-domain <domain>]`

Set GitHub Actions secrets for an app repo. Reads `SERVER_HOST`, `SERVER_USER`, and `SSH_KEY_PATH` from `~/.towlion.conf` and sets them as repo secrets along with `APP_DOMAIN`.

```
towlion secrets myapp app.example.com
towlion secrets myapp app.example.com --preview-domain example.com
```

Requires `gh` CLI to be installed and authenticated.

`towlion deploy <app>`

Trigger the deploy workflow via GitHub Actions (`gh workflow run`).

`towlion restart <app>`

Restart app containers. Automatically detects the active blue-green slot.

`towlion backup [app|all]`

Run a database backup. Defaults to all databases if no app specified.

`towlion rotate <app> [--type db|s3|all]`

Rotate credentials for an app without downtime. Defaults to rotating all credentials.

`towlion ssh`

SSH to the server as the deploy user.

`towlion tunnel <port>`

Open an SSH tunnel. Useful for accessing internal services:

```
towlion tunnel 5432   # PostgreSQL
towlion tunnel 3000   # Grafana
towlion tunnel 9000   # MinIO
```

`towlion verify`

Run `verify-server.sh` to check server health (30 checks).

`towlion alerts`

Run `check-alerts.sh` to check for active alerts.

4.2 Secrets Reference

A single-page reference for every secret and credential in the Towlion platform — where it lives, what it does, and how to rotate it.

4.2.1 GitHub Actions Secrets (per-app repo)

These are set in your app repository under **Settings > Secrets and variables > Actions**.

Secret	Required	Purpose	Example
`SERVER_HOST`	Yes	Server IP address	`203.0.113.10`
`SERVER_USER`	Yes	SSH user for deployment	`deploy`
`SERVER_SSH_KEY`	Yes	SSH private key (ed25519 or RSA)	Full PEM-encoded key
`APP_DOMAIN`	Yes	Production domain for the app	`app.example.com`
`PREVIEW_DOMAIN`	No	Base domain for preview environments	`example.com`

> **Tip**
>
> To reduce the number of required secrets from 4 to 1, use a self-hosted GitHub Actions runner instead.

4.2.2 Platform Bootstrap Variables

Passed as environment variables when running `bootstrap-server.sh`. See Self-Hosting for the full bootstrap process.

User-provided:

Variable	Required	Purpose	Default
`ACME_EMAIL`	Yes	Email for Let's Encrypt TLS certificates	—
`OPS_DOMAIN`	No	Domain for the Grafana dashboard	—
`ALERT_REPO`	No	GitHub `owner/repo` for alert issues (e.g., Trivy findings)	—
`ENABLE_METRICS`	No	Set `true` to start Prometheus, cAdvisor, node-exporter	—

Auto-generated by bootstrap (written to `/opt/platform/.env`, mode 600):

Variable	Purpose
`POSTGRES_PASSWORD`	PostgreSQL superuser password
`MINIO_ROOT_USER`	MinIO root username
`MINIO_ROOT_PASSWORD`	MinIO root password
`GRAFANA_ADMIN_PASSWORD`	Grafana admin password

These are generated with `openssl rand` during the first bootstrap run and are never stored outside the server.

4.2.3 Per-App Runtime Environment

Located at `/opt/apps/<name>/deploy/.env`. Created manually from `deploy/env.template` during initial app setup.

Required:

Variable	Purpose	Example
`APP_DOMAIN`	Application domain	`app.example.com`
`DATABASE_URL`	PostgreSQL connection string	`postgresql://user:pass@postgres:5432/app_db`
`REDIS_URL`	Redis connection string	`redis://redis:6379/0`

Optional:

Variable	Purpose
`S3_ENDPOINT`	MinIO/S3 endpoint URL
`S3_BUCKET`	Storage bucket name
`S3_ACCESS_KEY`	Storage access key
`S3_SECRET_KEY`	Storage secret key
`JWT_SECRET`	Secret key for JWT token signing
`CORS_ORIGINS`	Comma-separated allowed CORS origins
`EMAIL_API_KEY`	Transactional email API key
`EMAIL_FROM`	Sender email address

Note

The deploy workflow auto-injects `S3_*` and `JWT_SECRET` values from the per-app credentials file (if it exists), overwriting whatever is in `deploy/.env`. You do not need to manage these values manually after running `create-app-credentials.sh`.

4.2.4 Per-App Server Credentials

Located at `/opt/platform/credentials/<app-name>.env` (mode 600, owned by `deploy`).

Created by running:

```
bash /opt/platform/infrastructure/create-app-credentials.sh <app-name>
```

Variable	Purpose
`DB_USER`	Dedicated PostgreSQL user (`<app_name>_user`)
`DB_PASSWORD`	Password for the dedicated PostgreSQL user
`S3_ACCESS_KEY`	Dedicated MinIO access key (`<app-name>-user`)
`S3_SECRET_KEY`	MinIO secret key for the dedicated user
`JWT_SECRET`	Per-app JWT signing key

These credentials are injected into `deploy/.env` by `deploy-blue-green.sh` on every deploy.

4.2.5 Backup Encryption

Optional. Configured in `/opt/platform/.env`.

Variable	Purpose
`BACKUP_ENCRYPTION_KEY`	Path to a key file used for AES-256-CBC encryption of database backups

When set, `backup-postgres.sh` produces encrypted `.dump.enc` files. `restore-postgres.sh` and `verify-backup.sh` auto-detect encrypted backups and decrypt them. See Server Contract — Backup Encryption for details.

4.2.6 Developer CLI Config

Located at `~/.towlion.conf`. This is a local-only file — not a GitHub secret.

Variable	Purpose	Example
`SERVER_HOST`	Server IP or hostname	`203.0.113.10`
`SERVER_USER`	SSH user	`deploy`
`SSH_KEY_PATH`	Path to SSH private key	`~/.ssh/id_ed25519`

See CLI Reference for the full command list.

4.2.7 Rotation Reference

Secret	Rotation Method	Command / Runbook
Per-app DB password	Script	`rotate-credentials.sh <app> --type db`
Per-app S3 credentials	Script	`rotate-credentials.sh <app> --type s3`
Per-app JWT secret	Script	`rotate-credentials.sh <app> --type jwt`
All per-app credentials	Script	`rotate-credentials.sh <app> --type all`
Platform PostgreSQL password	Script	`rotate-credentials.sh --platform --type db`
Platform MinIO root password	Script	`rotate-credentials.sh --platform --type s3`
All platform credentials	Script	`rotate-credentials.sh --platform --type all`
SSH keys	Manual	Rotate Credentials runbook — SSH section
GitHub Actions secrets	Manual	Update in repo Settings > Secrets after rotating the corresponding server-side credential
Backup encryption key	Manual	Generate new key file, re-encrypt existing backups, update `BACKUP_ENCRYPTION_KEY` path

Tip

Add `--yes` to any `rotate-credentials.sh` command to skip the confirmation prompt. The script verifies app health after every rotation.

4.3 Observability

The Towlion platform provides a complete observability stack: structured logging, dashboards, and alerting. All apps inherit this automatically from the app-template.

4.3.1 Structured JSON Logging

Every Towlion app uses `python-json-logger` to emit structured JSON logs to stdout. Docker captures these, Promtail scrapes them, and Loki stores them for querying in Grafana.

What Gets Logged

The FastAPI middleware logs every HTTP request as a JSON object:

```
{
  "timestamp": "2026-03-16T12:00:00",
  "level": "INFO",
  "message": "request",
  "method": "GET",
  "path": "/health",
  "status_code": 200,
  "duration_ms": 1.23,
  "client_ip": "172.18.0.1"
}
```

How It Works

In `app/main.py`, the app-template configures:

1. **Root logger** with `StreamHandler(stdout)` and `JsonFormatter`
2. **Request middleware** that logs method, path, status_code, duration_ms, and client_ip

The Dockerfile uses `--no-access-log` to suppress uvicorn's default access log, since the structured middleware replaces it with richer data.

Adding Custom Log Lines

Use the standard Python `logging` module — it inherits the JSON formatter:

```
import logging
logger = logging.getLogger(__name__)
logger.info("task completed", extra={"task_id": 42, "result": "success"})
```

4.3.2 Dashboards

Platform Overview

UID: `towlion-platform-overview`

Shows all container logs, error rate by container, and a filterable log viewer. Available by default.

App Dashboard

UID: `towlion-app-dashboard`

Per-app observability with a `$app` dropdown:

- **Request Rate** — `count_over_time` by HTTP method (LogQL)
- **Error Rate (5xx)** — Count of responses with status_code >= 500 (LogQL)
- **Response Time p95** — `quantile_over_time` on `duration_ms` field (LogQL, requires Loki 3.0+)
- **Container CPU/Memory** — PromQL panels (requires metrics profile)
- **Recent Logs** — Filterable log stream

Resource Metrics

UID: `towlion-resource-metrics`

Host and container resource usage. Requires the metrics profile (`COMPOSE_PROFILES=metrics`). See Server Contract.

4.3.3 Alerting Rules

Three Grafana native alert rules are provisioned automatically:

Rule	Source	Condition	Severity
Error Rate Spike	LogQL	>10 HTTP 5xx errors in 5 minutes	Warning
Container Down	PromQL	Container stops reporting metrics	Critical
Disk Usage >85%	PromQL	Root filesystem above 85%	Warning

The Container Down and Disk Usage rules require the metrics profile to be enabled.

These rules supplement the existing `check-alerts.sh` cron job, which runs every 5 minutes and checks containers, disk, memory, TLS, restarts, backups, and HTTP health.

Viewing Alerts

Navigate to **Alerting > Alert rules** in Grafana (ops.anulectra.com) to see rule status, silences, and history.

4.3.4 LogQL Query Examples

All requests to a specific app:

```
{container=~"todo-app.*"} | json
```

Slow requests (>500ms):

```
{container=~"todo-app.*"} | json | duration_ms > 500
```

Error responses only:

```
{container=~"todo-app.*"} | json | status_code >= 400
```

Request count by path (last hour):

```
sum by (path) (count_over_time({container=~"todo-app.*"} | json | __error__=`` [1h]))
```

p99 response time:

```
quantile_over_time(0.99, {container=~"todo-app.*"} | json | unwrap duration_ms [5m]) by ()
```

4.3.5 Enabling Metrics

The CPU/Memory panels in the App Dashboard and two of the three alert rules require Prometheus, cAdvisor, and node-exporter. To enable:

```
# Add to /opt/platform/.env
echo "COMPOSE_PROFILES=metrics" >> /opt/platform/.env

# Restart services
cd /opt/platform && docker compose up -d
```

Or bootstrap with `sudo ENABLE_METRICS=true bash bootstrap-server.sh`

4.4 Troubleshooting

Symptom-based guide for diagnosing common platform issues.

4.4.1 Container Won't Start

Check:

```
docker compose -p <app-name> -f /opt/apps/<app-name>/deploy/docker-compose.yml logs --tail 50 app
docker compose -p <app-name> -f /opt/apps/<app-name>/deploy/docker-compose.yml ps
```

Common causes:

Cause	What you'll see	Fix
Missing env var	`KeyError` or `ValidationError` in logs	Add the variable to `deploy/.env`
Port conflict	`Address already in use`	Check `docker ps` for conflicting containers
Image build failure	Container stuck in `Created` state	Check build output in GitHub Actions logs
Import error	`ModuleNotFoundError`	Add the missing package to `requirements.txt`
Bad migration	`alembic.util.exc.CommandError`	Fix the migration and re-deploy

4.4.2 Caddy Certificate Provisioning Fails

Check:

```
docker compose -f /opt/platform/docker-compose.yml logs --tail 50 caddy
```

Common causes:

Cause	What you'll see	Fix
DNS not pointed	`no A/AAAA records found`	Create an A record pointing the domain to the server IP
Rate limit hit	`too many certificates`	Wait 1 hour (Let's Encrypt rate limit resets)
Port 80 blocked	`connection refused` on ACME challenge	Verify UFW allows port 80: `sudo ufw status`
Wrong domain in Caddyfile	Certificate for wrong domain	Check `/opt/platform/caddy-apps/<app>.caddy`

Verify DNS:

```
dig +short <app-domain>
# Should return your server IP
```

4.4.3 Database Connection Refused

Check:

```
docker exec platform-postgres-1 pg_isready
docker compose -f /opt/platform/docker-compose.yml ps postgres
```

Common causes:

Cause	What you'll see	Fix
PostgreSQL container down	`pg_isready` fails or container not `Up`	`cd /opt/platform && docker compose up -d postgres`
Wrong credentials	`password authentication failed` in app logs	Verify `DATABASE_URL` in `deploy/.env` matches credentials
Database doesn't exist	`database "<name>" does not exist`	The deploy workflow creates it automatically; re-run the deploy
Not on Docker network	`could not translate host name "postgres"`	Check app container is on the `towlion` network

4.4.4 App Returns 502 Bad Gateway

Check:

```
# Is the app container running?
docker ps --filter name=<app-name>

# What do the app logs say?
docker compose -p <app-name> -f /opt/apps/<app-name>/deploy/docker-compose.yml logs --tail 50 app

# What does the Caddyfile say?
cat /opt/platform/caddy-apps/<app-name>.caddy
```

Common causes:

Cause	What you'll see	Fix
App container crashed	Container not in `docker ps` output	Check logs, fix the crash, restart
App still starting	502 for a few seconds after deploy	Wait 5-10 seconds; if persistent, check logs
Wrong upstream in Caddyfile	Caddy can't reach the container	Verify container name matches Caddyfile (e.g., `<app-name>-app-1:8000`)
App listening on wrong port	Container running but not responding	Verify the app binds to `0.0.0.0:8000`

4.4.5 Preview Environment Not Accessible

Check:

```
# DNS resolves?
dig +short pr-<N>.preview.<app>.<domain>

# Caddyfile exists?
cat /opt/platform/caddy-apps/<app-name>-pr-<N>.caddy

# Container running?
docker ps --filter name=<app-name>-pr-<N>
```

Common causes:

Cause	What you'll see	Fix
Wildcard DNS not set	`dig` returns nothing	Add `*.preview.<app>.<domain>` A record pointing to server IP
Preview container crashed	Container not running	Check logs: `docker logs <app-name>-pr-<N>-app-1`
Missing `PREVIEW_DOMAIN` secret	Workflow skips preview deployment	Set `PREVIEW_DOMAIN` in repo secrets
PR already closed	Cleanup removed the container	Reopen the PR or push a new commit to trigger preview

4.4.6 Disk Space Full

Check:

```
df -h /
df -h /data
docker system df
du -sh /data/*
du -sh /var/log/*
```

Fix by priority:

1. **Docker cleanup** — remove unused images, containers, and build cache:

```
docker system prune -f
docker image prune -a -f  # removes ALL unused images
```

2. **Old backups** — backups older than 7 days should be auto-pruned, but check:

```
ls -lh /data/backups/postgres/
```

3. **Log files** — check for oversized log files:

```
du -sh /var/log/* | sort -rh | head -10
```

4. **Loki data** — if log storage is consuming significant space:

```
du -sh /data/loki/
```

Prevention: The `check-alerts.sh` cron job (every 5 minutes) creates a GitHub issue when disk usage exceeds 90%.

4.5 Runbooks

4.5.1 Runbook: Restart an Application

When to Use

- Application is unresponsive or returning errors
- After a configuration change in `deploy/.env`
- After manual database changes that require the app to reconnect

Steps

1. SSH INTO THE SERVER

```
ssh deploy@YOUR_SERVER_IP
```

2. NAVIGATE TO THE APP DIRECTORY

```
cd /opt/apps/<app-name>
```

3. RESTART THE APP CONTAINER

```
docker compose -p <app-name> -f deploy/docker-compose.yml restart app
```

To restart all containers for the app (including workers if present):

```
docker compose -p <app-name> -f deploy/docker-compose.yml restart
```

4. VERIFY THE CONTAINER IS RUNNING

```
docker compose -p <app-name> -f deploy/docker-compose.yml ps
```

All services should show `Up` status.

5. CHECK THE HEALTH ENDPOINT

```
curl -s https://<app-domain>/health
```

Expected response: `{"status": "healthy"}` with HTTP 200.

6. CHECK CONTAINER LOGS IF UNHEALTHY

```
docker compose -p <app-name> -f deploy/docker-compose.yml logs --tail 50 app
```

Full Rebuild

If a restart doesn't resolve the issue, rebuild the container:

```
cd /opt/apps/<app-name>
docker compose -p <app-name> -f deploy/docker-compose.yml up -d --build app
```

Notes

- Restarting an app causes brief downtime (typically a few seconds). Caddy returns 502 during this window.
- Restarting does **not** affect other applications on the server.
- Platform services (PostgreSQL, Redis, etc.) are managed separately in `/opt/platform/`.

4.5.2 Runbook: Add a New Application

When to Use

- Deploying a new application to the server
- Setting up a new repo from the app template

Prerequisites

- Server already bootstrapped (Self-Hosting Guide)
- DNS A record pointing your app domain to the server IP
- SSH access as the `deploy` user

Steps

1. CREATE THE REPOSITORY

Use GitHub's "Use this template" button on towlion/app-template to create a new repo under the `towlion` organization (or your own account).

2. PROVISION PER-APP CREDENTIALS ON THE SERVER

```
ssh deploy@YOUR_SERVER_IP
sudo bash /opt/platform/infrastructure/create-app-credentials.sh <app-name>
```

This creates an isolated PostgreSQL user, MinIO bucket, and writes credentials to `/opt/platform/credentials/<app-name>.env`.

3. CLONE THE REPO ON THE SERVER

```
ssh deploy@YOUR_SERVER_IP
cd /opt/apps
git clone https://github.com/<org>/<app-name>.git
```

4. CREATE `deploy/.env`

```
cd /opt/apps/<app-name>
cp deploy/env.template deploy/.env
```

Edit `deploy/.env` with your values. If you ran `create-app-credentials.sh`, the deploy workflow will automatically fill in the database and S3 credentials on first deploy.

5. CONFIGURE GITHUB SECRETS

Set these repository secrets (**Settings > Secrets and variables > Actions**):

Secret	Value
`SERVER_HOST`	Your server IP
`SERVER_USER`	`deploy`
`SERVER_SSH_KEY`	SSH private key for the deploy user
`APP_DOMAIN`	Your app's domain (e.g., `app.example.com`)

Optional:

Secret	Value
`PREVIEW_DOMAIN`	Base domain for PR previews (e.g., `example.com`). Also add a `*.preview.<app-name>.<domain>` DNS A record.

6. PUSH TO MAIN

Push code to the `main` branch. The deploy workflow will:

1. SSH into the server
2. Pull the latest code
3. Create the app database (if it doesn't exist)
4. Build and start containers
5. Run Alembic migrations
6. Write a Caddyfile and reload Caddy

7. VERIFY

```
curl -s https://<app-domain>/health
```

Expected: `{"status": "healthy"}` with HTTP 200.

References

- Deployment Tutorial — full step-by-step walkthrough
- Deploy Prompt — reusable Claude Code session prompt for deployments
- Server Contract — directory layout and lifecycle details

4.5.3 Runbook: Rotate Credentials

When to Use

- Scheduled credential rotation
- Suspected credential compromise
- After team member access changes

PostgreSQL Credentials

PER-APP DATABASE USER

```
ssh deploy@YOUR_SERVER_IP

# Generate a new password
NEW_PW=$(openssl rand -base64 24)

# Update the PostgreSQL user
docker exec -i platform-postgres-1 psql -U postgres -c \
  "ALTER ROLE <app_name>_user WITH PASSWORD '${NEW_PW}';"

# Update the credentials file
sed -i "s|^DB_PASSWORD=.*|DB_PASSWORD=${NEW_PW}|" \
  /opt/platform/credentials/<app-name>.env

# Update the app's deploy/.env
cd /opt/apps/<app-name>
sed -i "s|^DATABASE_URL=.*|DATABASE_URL=postgresql://<app_name>_user:${NEW_PW}@postgres:5432/<app_name>_db|" \
  deploy/.env

# Restart the app to pick up the new credentials
docker compose -p <app-name> -f deploy/docker-compose.yml restart app
```

PLATFORM POSTGRES SUPERUSER

```
NEW_PW=$(openssl rand -base64 24)

docker exec -i platform-postgres-1 psql -U postgres -c \
  "ALTER ROLE postgres WITH PASSWORD '${NEW_PW}';"

sed -i "s|^POSTGRES_PASSWORD=.*|POSTGRES_PASSWORD=${NEW_PW}|" \
  /opt/platform/.env

# Restart platform services
cd /opt/platform && docker compose up -d
```

Important: After rotating the platform postgres password, update `DATABASE_URL` in every app's `deploy/.env` that uses the superuser directly (apps without per-app credentials).

MinIO Credentials

PER-APP MINIO USER

```
NEW_KEY=$(openssl rand -base64 24)

# Update MinIO user via mc CLI
docker exec -i platform-minio-1 mc admin user update local <app-name>-user "${NEW_KEY}"

# Update the credentials file
sed -i "s|^S3_SECRET_KEY=.*|S3_SECRET_KEY=${NEW_KEY}|" \
  /opt/platform/credentials/<app-name>.env

# Update the app's deploy/.env
cd /opt/apps/<app-name>
sed -i "s|^S3_SECRET_KEY=.*|S3_SECRET_KEY=${NEW_KEY}|" deploy/.env

# Restart the app
docker compose -p <app-name> -f deploy/docker-compose.yml restart app
```

PLATFORM MINIO ROOT CREDENTIALS

```
NEW_KEY=$(openssl rand -base64 24)

sed -i "s|^MINIO_ROOT_PASSWORD=.*|MINIO_ROOT_PASSWORD=${NEW_KEY}|" \
  /opt/platform/.env
```

```
cd /opt/platform && docker compose up -d minio
```

Grafana Admin Password

```
NEW_PW=$(openssl rand -base64 24)

sed -i "s|^GRAFANA_ADMIN_PASSWORD=.*|GRAFANA_ADMIN_PASSWORD=${NEW_PW}|" \
  /opt/platform/.env

cd /opt/platform && docker compose up -d grafana
```

The new password takes effect on next Grafana restart. Existing browser sessions remain valid until they expire.

SSH Keys

ROTATE THE DEPLOY USER'S SSH KEY

1. Generate a new key pair locally:

```
ssh-keygen -t ed25519 -f ~/.ssh/deploy-new -C "deploy@towlion"
```

1. Add the new public key to the server:

```
ssh deploy@YOUR_SERVER_IP
echo "NEW_PUBLIC_KEY" >> ~/.ssh/authorized_keys
```

1. Test the new key:

```
ssh -i ~/.ssh/deploy-new deploy@YOUR_SERVER_IP echo "success"
```

1. Remove the old public key from `~/.ssh/authorized_keys` on the server.
2. Update the `SERVER_SSH_KEY` secret on **every app repository** that deploys to this server.

Post-Rotation Verification

After rotating any credential:

1. Check the app health endpoint: `curl https://<app-domain>/health`
2. Run a deploy to confirm the workflow still works
3. Check container logs for authentication errors:

```
docker compose -p <app-name> -f deploy/docker-compose.yml logs --tail 20 app
```

4.5.4 Runbook: Restore a Database Backup

When to Use

- Data corruption or accidental deletion
- Rolling back to a known-good state
- Migrating data between servers

Prerequisites

- SSH access as the `deploy` user
- Backups exist in `/data/backups/postgres/`

Steps

1. LIST AVAILABLE BACKUPS

```
ssh deploy@YOUR_SERVER_IP
ls -lh /data/backups/postgres/
```

Backups follow the naming pattern `<database>_YYYYMMDD_HHMMSS.sql.gz`. The backup cron runs daily at 02:00 UTC with 7-day retention.

2. CHOOSE A BACKUP

Identify the backup file to restore. Example:

```
todo_app_db_20260315_020001.sql.gz
```

3. RUN THE RESTORE SCRIPT

Interactive mode (prompts for confirmation):

```
bash /opt/platform/infrastructure/restore-postgres.sh
```

The script will list available backups and prompt you to select one.

Non-interactive mode (for scripting):

```
bash /opt/platform/infrastructure/restore-postgres.sh --yes <backup-file>
```

4. VERIFY THE RESTORED DATA

Connect to the database and check key tables:

```
docker exec -i platform-postgres-1 psql -U postgres -d <database_name> -c "\dt"
docker exec -i platform-postgres-1 psql -U postgres -d <database_name> -c "SELECT count(*) FROM <table_name>;"
```

5. RESTART THE APPLICATION

After restoring, restart the app to ensure it reconnects cleanly:

```
cd /opt/apps/<app-name>
docker compose -p <app-name> -f deploy/docker-compose.yml restart app
```

6. VERIFY THE APPLICATION

```
curl -s https://<app-domain>/health
```

Important Notes

- **Restoring drops the existing database** and recreates it from the backup. All data written after the backup was taken will be lost.
- Backups are retained for 7 days. If you need a backup older than 7 days, you must have configured off-server backup sync (e.g., via `rclone`).
- The restore script operates on the platform PostgreSQL instance. It does not affect other databases on the same instance.
- Consider stopping the application before restoring to avoid write conflicts:

```
docker compose -p <app-name> -f deploy/docker-compose.yml stop app
# ... restore ...
docker compose -p <app-name> -f deploy/docker-compose.yml start app
```

4.5.5 Runbook: Debug a Failed Deployment

When to Use

- GitHub Actions deploy workflow fails
- App is not accessible after a deploy
- Containers are crashing after deploy

Diagnostic Steps

1. CHECK GITHUB ACTIONS LOGS

Go to the repository's **Actions** tab and open the failed workflow run. Check both jobs:

- **test** — build and test failures (Dockerfile issues, missing dependencies, test failures)
- **deploy** — SSH or server-side failures

Look for the first red step. The error message usually points directly to the cause.

2. SSH INTO THE SERVER

```
ssh deploy@YOUR_SERVER_IP
```

3. CHECK CONTAINER STATUS

```
docker compose -p <app-name> -f /opt/apps/<app-name>/deploy/docker-compose.yml ps
```

Look for containers in `Restarting`, `Exited`, or `Created` (not `Up`) states.

4. CHECK CONTAINER LOGS

```
docker compose -p <app-name> -f /opt/apps/<app-name>/deploy/docker-compose.yml logs --tail 100 app
```

Common error patterns:

- `ModuleNotFoundError` — missing Python dependency in `requirements.txt`
- `Connection refused` to postgres — database not reachable or wrong credentials
- `Address already in use` — port conflict with another container
- `PermissionError` — file ownership issue in mounted volumes

5. CHECK DISK SPACE AND MEMORY

```
df -h /
free -h
docker system df
```

If disk is above 90%, clean up:

```
docker system prune -f
```

6. VERIFY THE DOCKER NETWORK

```
docker network inspect towlion
```

Confirm the app container and platform services (postgres, redis, caddy) are all on the `towlion` network.

7. VERIFY `deploy/.env`

```
cat /opt/apps/<app-name>/deploy/.env
```

Check that `DATABASE_URL`, `SECRET_KEY`, and other required variables are set. Compare against `deploy/env.template` for any missing values.

8. CHECK THE CADDYFILE

```
cat /opt/platform/caddy-apps/<app-name>.caddy
```

Verify: - The domain matches `APP_DOMAIN` - The upstream container name is correct (e.g., `<app-name>-app-1:8000`) - The `security_headers` snippet is imported

Reload Caddy if you made changes:

```
docker compose -f /opt/platform/docker-compose.yml exec -T caddy caddy reload --config /etc/caddy/Caddyfile
```

Common Causes

Symptom	Likely Cause	Fix
SSH connection refused	Wrong `SERVER_HOST` or SSH key	Verify GitHub secret matches server IP and key
Docker build fails	Missing dependency or syntax error	Fix `Dockerfile` or `requirements.txt`
Container exits immediately	Missing env var or import error	Check `deploy/.env` and container logs
502 Bad Gateway	App container not running or wrong upstream	Check `docker ps` and Caddyfile
Alembic migration fails	Schema conflict or missing migration	SSH in and run `alembic upgrade head` manually to see the error
"Network not found"	`towlion` network missing	Run `docker network create towlion`
Disk full	Old images/containers accumulating	`docker system prune -f`
Permission denied	File ownership mismatch	Check that `deploy` user owns `/opt/apps/<app-name>`

After Fixing

1. Re-run the deploy workflow from the GitHub Actions tab (click "Re-run all jobs")
2. Or trigger a new deploy by pushing a commit
3. Verify the health endpoint: `curl -s https://<app-domain>/health`

5. Project

5.1 Ecosystem

5.1.1 GitHub Organization as Application Catalog

The Towlion GitHub organization serves as a catalog of deployable applications. Each repository is a standalone, self-hostable application.

```
towlion/
  .github              # Organization profile, community health files, and reusable workflows
  platform             # Architecture docs and platform tools
  infra                # Infrastructure provisioning CLI (OpenTofu)
  app-template         # Template for bootstrapping new apps
  todo-app             # Todo CRUD application
  wit                  # Work Item Tracker (Kanban board)
```

5.1.2 Repository Types

Organization Profile (`towlion/.github`)

The special `.github` repository provides the organization-level profile README displayed on github.com/towlion. Contains:

- `profile/README.md` — marketing content shown on the org landing page

Platform Repository (`towlion/platform`)

The meta repository for the ecosystem. Contains:

- Architecture documentation
- Platform specification
- Deployment guides
- Developer CLI (`cli/towlion`) — see CLI reference
- Infrastructure scripts (blue-green deploy, backup verification, credential rotation)

Infrastructure CLI (`towlion/infra`)

CLI tool for provisioning Towlion servers on DigitalOcean or AWS using OpenTofu. Creates a Debian server with:

- Firewall (ports 22, 80, 443)
- 50 GB data volume mounted at `/data`
- SSH key pair
- Optional DNS zone with root and wildcard A records

Usage: `./towlion-infra init --provider <aws|digitalocean>` then `./towlion-infra apply`.

Application Template (`towlion/app-template`)

A GitHub Template Repository for creating new applications. Includes:

- FastAPI backend scaffold
- Next.js frontend scaffold
- Docker configuration
- GitHub Actions deployment workflow
- Database migration setup

New apps are created by:

1. Click "Use this template" to create a new repo
2. Customize the application code
3. Push — deploys automatically

Application Repositories

Each application is a **separate GitHub repository** under the `towlion` organization. Each one:

- Is a standalone web application in its own repo
- Contains its own Docker configuration and deployment workflow
- Can be forked and self-hosted independently
- Follows the application specification
- Follows the platform's governance policies, including branch protection, PR requirements, and commit conventions

5.1.3 Multi-Application Runtime

The platform supports running many applications on a single server. All applications share core infrastructure:

```
Server
  ├── Caddy (reverse proxy)
  ├── PostgreSQL
  ├── Redis
  ├── MinIO
  ├── App 1 container
  ├── App 2 container
  └── App 3 container
```

Each application gets:

- Its own subdomain (e.g., `uku.towlion.com`)
- Its own database (e.g., `uku_db`)
- Its own storage bucket
- Its own container(s)

For details on how the server is structured and how app workflows interact with it, see Server Contract.

5.1.4 Core Philosophy

1. Repository-Driven Deployment

Each repository deploys itself — no central deployment platform needed.

```
repository → GitHub Actions → deployed application
```

2. Forkable Infrastructure

Anyone can deploy by forking:

```
fork → configure secrets → push → running application
```

3. Single-Server SaaS

Optimized for indie developers, small SaaS apps, AI-generated tools, and personal product ecosystems.

5.1.5 Target Use Cases

The ecosystem is designed for:

- Small SaaS applications
- Open-source developer tools
- Web utilities
- AI-generated applications
- Self-hosted services

5.2 Governance

This document is the single source of truth for repository policies across the Towlion organization.

5.2.1 Org-Level Rulesets

Org-level rulesets are configured manually in the GitHub UI. They apply to all repositories in the organization.

Ruleset: `towlion-main-protection`

Target: All repositories, default branch (`main`)

Rules:

- Require a pull request before merging (1 approval minimum)
- Require status checks to pass (`validate`)
- Block force pushes on `main`
- Block deletions of `main`

How to Configure

1. Go to the Towlion org settings
2. Navigate to **Settings > Rules > Rulesets**
3. Click **New ruleset > New branch ruleset**
4. Set the name to `towlion-main-protection`
5. Under **Target branches**, select **Default branch**
6. Enable **Restrict deletions**
7. Enable **Require a pull request before merging** — set minimum approvals to `1`
8. Enable **Require status checks to pass** — add `validate` as a required check
9. Enable **Block force pushes**
10. Set enforcement status to **Active**
11. Click **Create**

5.2.2 Repository Settings

These settings are applied automatically by the setup script.

Setting	Value
Default branch	`main`
Wiki	Disabled
Projects	Disabled
Discussions	Disabled
Auto-delete head branches	Enabled
Allowed merge types	Squash merge only

5.2.3 Branch Protection

Branch protection rules complement the org-level rulesets and are applied by the setup script.

Rules applied to `main`:

- Require pull request reviews (1 reviewer minimum)
- Dismiss stale reviews when new commits are pushed
- Require the `validate` status check to pass
- Require the branch to be up to date before merging
- No force pushes
- No deletions

5.2.4 CODEOWNERS

Every application repository must include a `.github/CODEOWNERS` file. At minimum:

```
* @towlion/maintainers
```

See the app-template CODEOWNERS for the canonical version.

5.2.5 PR Template

Every application repository must include a `.github/PULL_REQUEST_TEMPLATE.md` with the following sections:

- **Summary** — What this PR does and why
- **Changes** — List of changes made
- **Testing** — How the changes were tested
- **Checklist** — Standard checklist items

See the app-template PR template for the canonical version.

5.2.6 Required Labels

All application repositories must have the following labels. The setup script creates these automatically from `scripts/labels.json`.

Label	Color	Description
`bug`	`#d73a4a`	Something isn't working
`feature`	`#a2eeef`	New feature or request
`docs`	`#0075ca`	Documentation improvements
`chore`	`#e4e669`	Maintenance and housekeeping
`breaking`	`#b60205`	Breaking change
`good first issue`	`#7057ff`	Good for newcomers

5.2.7 Commit Conventions

Towlion uses simplified Conventional Commits.

Format

```
type: description
```

- Use **lowercase**
- Use **imperative mood** ("add feature" not "added feature")
- Keep the subject under **72 characters**

Types

Type	Purpose
`feat`	New feature
`fix`	Bug fix
`docs`	Documentation changes
`chore`	Maintenance and housekeeping
`refactor`	Code restructuring (no behavior change)
`test`	Adding or updating tests
`ci`	CI/CD changes

Optional Scope

```
feat(api): add health endpoint
fix(auth): handle expired tokens
```

Breaking Changes

Append `!` after the type:

```
feat!: change response format
```

Examples

```
feat: add user registration endpoint
fix(db): prevent duplicate migration runs
docs: update self-hosting guide
chore: upgrade dependencies
ci: add preview environment workflow
```

5.3 Onboarding

Getting started as a contributor or operator of the Towlion platform.

5.3.1 Prerequisites

Tool	Version	Purpose
Git	2.x+	Version control
Docker	24+	Container runtime
Docker Compose	v2+	Service orchestration
Python	3.11+	Backend development, validator
Node.js	20+	Frontend development
GitHub CLI (`gh`)	2.x+	Repository management

5.3.2 New Contributor Checklist

1. Understand the platform

- [ ] Read the Architecture Overview — how the platform works
- [ ] Read the App Specification — the contract apps must follow
- [ ] Read the Scope and Design Boundaries — what the platform does and doesn't do

2. Set up a development environment

- [ ] Fork or clone the app-template repository
- [ ] Install Python dependencies: `pip install -r requirements.txt`
- [ ] Run the app locally: `uvicorn app.main:app --reload --port 8000`
- [ ] Verify the health endpoint: `curl http://localhost:8000/health`

3. Run the spec validator

The platform repo includes a validator that checks apps against the spec:

```
git clone https://github.com/towlion/platform.git
cd platform
python validator/validate.py /path/to/your-app
```

All three tiers should pass before deploying.

4. Deploy to a server

- [ ] Bootstrap a server (Self-Hosting Guide) or get access to the test server
- [ ] Configure GitHub secrets: `SERVER_HOST`, `SERVER_USER`, `SERVER_SSH_KEY`, `APP_DOMAIN`
- [ ] Push to `main` and verify the deploy workflow succeeds
- [ ] Check the app at `https://<your-domain>/health`

5. Review operational docs

- [ ] Server Contract — directory layout, lifecycle, security
- [ ] Governance — commit conventions, branch protection, PR process
- [ ] Troubleshooting — common issues and fixes
- [ ] Runbooks — step-by-step operational procedures

5.3.3 Key Concepts

Concept	Description
GitHub as control plane	No custom dashboard — repos, Actions, and secrets manage everything
Fork-based self-hosting	Fork a repo, configure secrets, push to deploy on your own server
Shared infrastructure	PostgreSQL, Redis, MinIO, Caddy run once and serve all apps
Per-app isolation	Each app gets its own database, storage bucket, and container
Spec conformance	All apps follow the same contract (ports, endpoints, env vars)

5.4 Decisions

5.4.1 ADR 001: GitHub as the Control Plane

Date: 2025-12-01 **Status:** Accepted

Context

Traditional PaaS platforms (Heroku, Render, Fly.io) require a dedicated control plane — a custom API, dashboard, or CLI that manages deployments, configuration, and access control. Building and maintaining a control plane is a significant engineering effort and introduces another system to secure and operate.

Towlion targets indie developers and small projects. The overhead of a custom control plane would outweigh its benefits for this audience.

Decision

Use GitHub as the control plane for all platform operations:

- **CI/CD**: GitHub Actions workflows handle building, testing, and deploying
- **Configuration**: GitHub Secrets store deployment credentials
- **Access control**: GitHub repository permissions manage who can deploy
- **Workflow orchestration**: Actions workflows coordinate multi-step deployments
- **Issue tracking**: GitHub Issues for alerts (created by `check-alerts.sh`)

No custom dashboard, API server, or CLI is needed. The deployment flow is:

```
GitHub repository -> GitHub Actions -> SSH -> Docker runtime
```

Consequences

Benefits:

- Zero infrastructure to build or maintain for the control plane
- Familiar interface — developers already use GitHub daily
- Built-in audit trail via commit history and Actions logs
- Free for public repos; generous free tier for private repos
- Access control, 2FA, and SSO handled by GitHub

Trade-offs:

- Vendor dependency on GitHub (Actions, Secrets, API)
- Limited to what GitHub Actions can express (no custom UI for deployments)
- Secrets management is per-repository, not centralized
- No real-time deployment dashboard — must check Actions tab for status

5.4.2 ADR 002: Caddy Over Nginx for Reverse Proxy

Date: 2025-12-01 **Status:** Accepted

Context

The platform needs a reverse proxy to route traffic to application containers and handle TLS certificate provisioning. The two main contenders are Nginx (with certbot/Let's Encrypt) and Caddy.

Decision

Use Caddy as the reverse proxy.

Consequences

Benefits:

- **Automatic TLS** — Caddy provisions and renews Let's Encrypt certificates with zero configuration. No certbot cron jobs, no manual renewal scripts.
- **Simple configuration** — A Caddyfile is shorter and more readable than equivalent Nginx config. Adding a new app route is a 3-line file.
- **Per-app config fragments** — The `import /etc/caddy/apps/*.caddy` pattern allows deploy workflows to write individual `.caddy` files per app without editing a monolithic config.
- **Hot reload** — `caddy reload` applies config changes without dropping connections. No `nginx -s reload` dance.
- **Single binary** — No module system, no package dependencies. The official Docker image works out of the box.

Trade-offs:

- Nginx has broader community knowledge and more Stack Overflow answers
- Nginx supports more advanced configurations (TCP/UDP proxying, complex rewrites)
- Caddy is less battle-tested at high scale (not a concern for single-server deployments)
- Some hosting tutorials assume Nginx, requiring translation to Caddy equivalents

5.4.3 ADR 003: Single-Server Architecture

Date: 2025-12-01 **Status:** Accepted

Context

Modern deployment platforms typically use container orchestration (Kubernetes, Docker Swarm, Nomad) to distribute workloads across multiple servers. This provides high availability, auto-scaling, and fault tolerance — but at significant operational complexity.

Towlion targets indie developers, small SaaS products, and hobby projects. These applications typically serve a handful of users and run comfortably on a single server.

Decision

Limit the platform to a single Debian server. All applications, databases, and infrastructure services run on one machine using Docker Compose.

Consequences

Benefits:

- **Simplicity** — no cluster management, no service mesh, no distributed consensus
- **Low cost** — a single $12-24/month VPS runs the entire platform
- **Easy debugging** — everything is on one machine; `docker logs` and `docker exec` are sufficient
- **Fast deploys** — no image registry push/pull; images build locally from source
- **Predictable** — no network partitions, no node failures, no split-brain scenarios

Trade-offs:

- Single point of failure — if the server goes down, all apps go down
- No auto-scaling — resource limits are static
- Brief downtime during deploys (container rebuild takes a few seconds)
- Vertical scaling only — must move to a bigger server when capacity is reached
- Not suitable for high-traffic applications or strict uptime requirements

These are intentional design boundaries, not missing features. For workloads that need HA or auto-scaling, use Kubernetes, Fly.io, or a cloud-native platform instead.

5.4.4 ADR 004: Fork-Based Self-Hosting

Date: 2025-12-01 **Status:** Accepted

Context

Self-hosted software typically uses one of these distribution models:

1. **Installer script** — download and run a setup script (e.g., `curl | bash`)
2. **Docker image** — pull a pre-built image from a registry
3. **Helm chart / Terraform module** — declarative infrastructure definition
4. **Fork** — fork the repository and deploy from your own copy

Each model has different trade-offs for isolation, customization, and update management.

Decision

Use the fork model for self-hosting. Each operator forks the application repository, configures their own GitHub Secrets, and deploys from their fork.

```
Original repo -> fork by Alice -> alice.example.com
                -> fork by Bob   -> bob.example.com
                -> fork by Carol -> carol.example.com
```

Consequences

Benefits:

- **Strong isolation** — each fork is completely independent; no shared tenancy
- **Full customization** — operators can modify any code, workflow, or configuration
- **No installer to maintain** — the repository is the installer
- **GitHub handles updates** — operators can sync upstream changes via GitHub's fork sync
- **Built-in CI/CD** — GitHub Actions workflows come with the fork
- **Transparent** — all deployment logic is visible in the repository

Trade-offs:

- Operators must manage their own GitHub Secrets and server
- Upstream updates require manual sync (GitHub's "Sync fork" button)
- No centralized management across forks — each is fully independent
- Requires GitHub account and understanding of fork workflows
- Private forks require a GitHub paid plan

5.4.5 ADR 005: AppArmor Over SELinux

Date: 2026-03-01 **Status:** Accepted

Context

Mandatory Access Control (MAC) adds a security layer beyond standard Unix permissions. The two main MAC systems on Linux are AppArmor and SELinux. The platform runs on Debian 12, which ships with AppArmor enabled by default.

Decision

Use AppArmor (Debian's native MAC system) instead of SELinux.

Docker automatically applies the `docker-default` AppArmor profile to all containers, which restricts capabilities like writing to `/proc` and `/sys`, mounting filesystems, and accessing raw sockets. No additional configuration is needed.

Consequences

Benefits:

- **Zero configuration** — AppArmor is already active on Debian 12 out of the box
- **Docker integration** — Docker applies the `docker-default` profile automatically to every container
- **Debian-native** — maintained by the Debian security team, well-tested with Debian packages
- **Simple profile model** — AppArmor profiles are path-based and easier to read than SELinux policies

Trade-offs:

- AppArmor is less granular than SELinux (path-based vs. label-based)
- SELinux is the standard on RHEL/Fedora ecosystems; cross-distro documentation often assumes SELinux

Why not SELinux on Debian:

- SELinux policies on Debian are incomplete — the `selinux-policy-default` package lags far behind RHEL equivalents
- Docker + SELinux on Debian causes bind-mount labeling issues (`:z` / `:Z` volume flags) with no community support
- Enabling SELinux requires disabling AppArmor, losing Docker's automatic profile enforcement
- The Debian security team does not maintain SELinux policies to the same standard as AppArmor